

```

1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 | |-- latitude: double (nullable = true)
10 | |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: long (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_miliseconds: long (nullable = true)
16 |-- humidity: long (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)

```

```

1 %scala
2 display(spark.read.json(jsonSchema1Path).limit(10))

```

alarm_status	battery_level	c02_level	cca2	cca3	cn	coordinates	date	device_id	device_serial_num
1 yellow	3	1082	US	USA	United States	["latitude": 38, "longitude": -97]	2016-03-20	62	62fH8Kr8aIT
2 green	0	920	US	USA	null	["latitude": 47, "longitude": 8]	2016-03-20	241	241un29KmR
3 yellow	7	1004	US	USA	United States	["latitude": 42.36, "longitude": -71.05]	2016-03-20	260	260F7DTEbnz
4 yellow	9	1081	DE	DEU	Germany	["latitude": 51.45, "longitude": 7.02]	2016-03-20	298	298hJceQv0
5 yellow	8	1311	US	USA	United States	["latitude": 38.88, "longitude": -92.4]	2016-03-20	301	301wHcyNzw
6 red	1	1484	IN	IND	India	["latitude": 20, "longitude": 77]	2016-03-20	334	334DUAg4mbXI
7 yellow	9	1266	US	USA	null	["latitude": 47, "longitude": 8]	2016-03-20	349	349gRZSGtcGR
8 yellow	1	1085	GB	GBR	United Kingdom	["latitude": 51.51, "longitude": -0.09]	2016-03-20	383	383yNMmfRq0zG
9 red	3	1508	GB	GBR	United Kingdom	["latitude": 53.5, "longitude": -2.19]	2016-03-20	391	391Qn3ILA
10 yellow	5	1191	KR	KOR	Republic of Korea	["latitude": 37.29, "longitude": 127.01]	2016-03-20	455	455L4J9FUG

SCHEMA 2

```

1 %scala
2 // NEW => device_serial_number_device_type, location
3 spark.read.json(jsonSchema2Path).printSchema

```

```

1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- date: string (nullable = true)
6 |-- device_id: long (nullable = true)
7 |-- device_serial_number_device_type: string (nullable = true)
8 |-- epoch_time_miliseconds: long (nullable = true)
9 |-- humidity: double (nullable = true)
10 |-- ip: string (nullable = true)
11 |-- latitude: double (nullable = true)
12 |-- location: struct (nullable = true)
13 | |-- cca2: string (nullable = true)
14 | |-- cca3: string (nullable = true)
15 | |-- cn: string (nullable = true)
16 |-- longitude: double (nullable = true)
17 |-- scale: string (nullable = true)
18 |-- temp: double (nullable = true)
19 |-- timestamp: string (nullable = true)

```

```

1 %scala
2 display(spark.read.json(jsonSchema1Path).limit(10))

```

device_serial_number_device_type	epoch_time_miliseconds	humidity	ip	latitude	location	longitude	scale
1 {"2n2Pea": "sensor-pad"}	1458444054119	70	213.161.254.1	62.47	["cca2": "NO", "cca3": "NOR", "cn": "Norway"]	6.15	Fahre
2 {"12Y2klm0o": "sensor-pad"}	1458444054126	92	68.28.91.22	38	["cca2": "US", "cca3": "USA", "cn": "United States"]	-97	Fahre
3 {"230lupA": "meter-gauge"}	1458444054133	47	59.90.65.1	12.98	["cca2": "IN", "cca3": "IND", "cn": "India"]	77.58	Fahre
4 {"36VQv8fnEg": "sensor-pad"}	1458444054141	47	213.7.14.1	35	["cca2": "CY", "cca3": "CYP", "cn": "Cyprus"]	33	Fahre
5 {"448DeWGL": "sensor-pad"}	1458444054149	63	62.128.16.74	49.46	["cca2": "DE", "cca3": "DEU", "cn": "Germany"]	11.1	Fahre
6 {"49YesGXwt": "meter-gauge"}	1458444054162	70	170.37.224.1	42.28	["cca2": "US", "cca3": "USA", "cn": "United States"]	-71.44	Fahre
7 {"64djcin": "sensor-pad"}	1458444054162	55	38.99.198.186	38	["cca2": "US", "cca3": "USA", "cn": "United States"]	-97	Fahre
8 {"77IKW3YAB55": "meter-gauge"}	1458444054169	82	218.248.255.30	12.98	["cca2": "IN", "cca3": "IND", "cn": "India"]	77.58	Fahre
9 {"80TY4dWSMH": "sensor-pad"}	1458444054171	57	159.128.0.181	50.01	["cca2": "CA", "cca3": "CAN", "cn": "Canada"]	-97.22	Fahre
10 {"94HL9jCh": "sensor-pad"}	1458444054179	66	24.32.26.1	39.33	["cca2": "US", "cca3": "USA", "cn": "United States"]	-120.25	Fahre

EXAMPLE 1: SCHEMA TRACKING/MANAGEMENT

AL tracks schema versions, metadata and changes to input data over time via specifying a location directory path. These features are incredibly useful for tracking history of data lineage, and are tightly integrated with the Delta Lake transactional log **DESCRIBE HISTORY** and Time Travel.

```

1 %scala
2 val rawAlDf = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .option("cloudFiles.schemaLocation", repoSchemaPath) // schema history tracking
6   .load(jsonSchema1Path)
7 )

```

```

1 %scala
2 rawAlDf.printSchema

```

```

1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: string (nullable = true)
4 |-- c02_level: string (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: string (nullable = true)
9 |-- date: string (nullable = true)
10 |-- device_id: string (nullable = true)
11 |-- device_serial_number: string (nullable = true)
12 |-- device_type: string (nullable = true)
13 |-- epoch_time_milliseconds: string (nullable = true)
14 |-- humidity: string (nullable = true)
15 |-- ip: string (nullable = true)
16 |-- scale: string (nullable = true)
17 |-- temp: string (nullable = true)
18 |-- timestamp: string (nullable = true)
19 |-- _rescued_data: string (nullable = true)

```

```

1 %scala
2 display(rawAlDf.limit(10))

```

By default (for JSON, CSV and XML file format) AL infers all column data types as strings, including nested fields.

alarm_status	battery_level	c02_level	cca2	cca3	cn	coordinates	date	device_id	device_serial_
1 yellow	3	1082	US	USA	United States	("latitude":38.0,"longitude":-97.0)	2016-03-20	62	62IH8oKr8ajT
2 green	0	920	US	USA	null	("latitude":47.0,"longitude":8.0)	2016-03-20	241	241un29Kmr
3 yellow	7	1004	US	USA	United States	("latitude":42.36,"longitude":-71.05)	2016-03-20	260	260F7DTEbnz
4 yellow	9	1081	DE	DEU	Germany	("latitude":51.45,"longitude":7.02)	2016-03-20	298	298hJceoQv0
5 yellow	8	1311	US	USA	United States	("latitude":38.88,"longitude":-92.4)	2016-03-20	301	301wHcyNzw
6 red	1	1484	IN	IND	India	("latitude":20.0,"longitude":77.0)	2016-03-20	334	334DUAg4mb
7 yellow	9	1266	US	USA	null	("latitude":47.0,"longitude":8.0)	2016-03-20	349	349gRZSGicG
8 yellow	1	1085	GB	GBR	United Kingdom	("latitude":51.51,"longitude":-0.09)	2016-03-20	383	383yNMmfrQ0
9 red	3	1508	GB	GBR	United Kingdom	("latitude":53.5,"longitude":-2.19)	2016-03-20	391	391Qn3lLA
10 yellow	5	1191	KR	KOR	Republic of Korea	("latitude":37.29,"longitude":127.01)	2016-03-20	455	455L4J9FUG

Here is the directory structure where AL stores schema versions. These files can be read via **Spark DataFrame API**.

Schema Repository

```

1 %scala
2 display(dbutils.fs.ls(repoSchemaPath + "/_schemas"))

```

path	name	size	modificationTime
1 dbfs:/user/garrett.peternel@databricks.com/repo/lot-ddl/json/_schemas/0	0	1628	1709306037000

Schema Metadata

```
1 %scala
2 display(spark.read.json(repoSchemaPath + "/_schemas"))
```

Table < + New result table: OFF

	_corrupt_record	dataSchemaJson	partitionSchemaJson
1	v1	null	null
2		{ "type": "struct", "fields": { ("name": "alarm_status", "type": "string", "nullable": true, "metadata": {}), ("name": "battery_level", "type": "long", "nullable": true, "metadata": {}), ("name": "c02_level", "type": "long", "nullable": true, "metadata": {}), ("name": "cca2", "type": "string", "nullable": true, "metadata": {}), ("name": "cca3", "type": "string", "nullable": true, "metadata": {}), ("name": "cn", "type": "string", "nullable": true, "metadata": {}), ("name": "coordinates", "type": "struct", "fields": { ("name": "latitude", "type": "do...	{ "type": "struct", "fields": {} }

EXAMPLE 2: SCHEMA HINTS

AL provides hint logic using SQL DDL syntax to enforce and override dynamic schema inference on known single data types, as well as semi-structured complex data types.

```
1 %scala
2 val hintAldf = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .option("cloudFiles.schemaLocation", repoSchemaPath)
6   .option("cloudFiles.schemaHints", "coordinates STRUCT<latitude:DOUBLE,
7     longitude:DOUBLE>, humidity LONG, temp DOUBLE") // schema ddl hints
8   .load(jsonSchema1Path)
9   )
```

```
1 %scala
2 hintAldf.printSchema
```

```
1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: string (nullable = true)
4 |-- c02_level: string (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 |   |-- latitude: double (nullable = true)
10 |   |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: string (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_milliseconds: string (nullable = true)
16 |-- humidity: long (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)
21 |-- _rescued_data: string (nullable = true)
```

The schema hints specified in the AL options perform the data type mappings on the respective columns. Hints are useful for applying schema enforcement on portions of the schema where data types are known while in tandem with dynamic schema inference covered in Example 3.

```
1 %scala
2 display(hintAldf.limit(10))
```

Table < + New result table: OFF

	coordinates	date	device_id	device_serial_number	device_type	epoch_time_milliseconds	humidity	ip	scale
1	{ "latitude": 38, "longitude": -97 }	2016-03-20	62	62fH8oK8aiT	sensor-pad	1458444054161	31	151.198.215.1	Fahrenheit
2	{ "latitude": 47, "longitude": 8 }	2016-03-20	241	241un29KmR	meter-gauge	1458444054259	64	80.84.21.105	Fahrenheit
3	{ "latitude": 42.36, "longitude": -71.05 }	2016-03-20	260	260F7DTEbnz	sensor-pad	1458444054267	57	72.21.168.109	Fahrenheit
4	{ "latitude": 51.45, "longitude": 7.02 }	2016-03-20	298	298hJceoQv0	sensor-pad	1458444054280	84	217.76.103.233	Fahrenheit
5	{ "latitude": 38.86, "longitude": -92.4 }	2016-03-20	301	301wHcyNzw	meter-gauge	1458444054281	26	150.199.7.154	Fahrenheit
6	{ "latitude": 20, "longitude": 77 }	2016-03-20	334	334DUAg4mbXi	sensor-pad	1458444054292	72	202.71.135.89	Fahrenheit
7	{ "latitude": 47, "longitude": 8 }	2016-03-20	349	349gRZSGtcGR	meter-gauge	1458444054296	45	195.219.212.9	Fahrenheit
8	{ "latitude": 51.51, "longitude": -0.09 }	2016-03-20	383	383yNMmIRq0zG	meter-gauge	1458444054308	46	166.49.222.157	Fahrenheit
9	{ "latitude": 53.5, "longitude": -2.19 }	2016-03-20	391	391Qn3iLA	meter-gauge	1458444054310	97	89.21.16.18	Fahrenheit
10	{ "latitude": 37.29, "longitude": 127.01 }	2016-03-20	455	A55L4J9FUG	therm-stick	1458444054330	64	211.41.134.73	Fahrenheit

EXAMPLE 3: DYNAMIC SCHEMA INFERENCE

AL dynamically searches a sample of the dataset to determine nested structure. This avoids costly and slow full dataset scans to infer schema. The following configurations are available to adjust the amount of sample data used on read to discover initial schema:

1. `spark.databricks.cloudFiles.schemaInference.sampleSize.numBytes` (default 50 GB)
2. `spark.databricks.cloudFiles.schemaInference.sampleSize.numFiles` (default 1000 files)

```

1 %scala
2 val inferAlDf = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .option("cloudFiles.schemaLocation", repoSchemaPath)
6   .option("cloudFiles.inferColumnTypes", true) // schema inference
7   .load(jsonSchema1Path)
8 )

```

```

1 %scala
2 inferAlDf.printSchema

```

```

1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 |   |-- latitude: double (nullable = true)
10 |   |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: long (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_milliseconds: long (nullable = true)
16 |-- humidity: long (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)
21 |-- _rescued_data: string (nullable = true)

```

AL saves the initial schema to the schema location path provided. This schema serves as the base version for the stream during incremental processing. Dynamic schema inference is an automated approach to applying schema changes over time.

```

1 %scala
2 display(inferAlDf.limit(10))

```

	alarm_status	battery_level	c02_level	cca2	cca3	cn	coordinates	date	device_id	device_serial_num
1	yellow	3	1082	US	USA	United States	["latitude": 38, "longitude": -97]	2016-03-20	62	62fH8oK8aiT
2	green	0	920	US	USA	null	["latitude": 47, "longitude": 8]	2016-03-20	241	241un29KmR
3	yellow	7	1004	US	USA	United States	["latitude": 42.36, "longitude": -71.05]	2016-03-20	260	260F7DTEbnz
4	yellow	9	1081	DE	DEU	Germany	["latitude": 51.45, "longitude": 7.02]	2016-03-20	298	298hJceoQv0
5	yellow	8	1311	US	USA	United States	["latitude": 38.88, "longitude": -92.4]	2016-03-20	301	301wHcyNzw
6	red	1	1484	IN	IND	India	["latitude": 20, "longitude": 77]	2016-03-20	334	334DUAg4mbXi
7	yellow	9	1266	US	USA	null	["latitude": 47, "longitude": 8]	2016-03-20	349	349gRZSGtcGR
8	yellow	1	1085	GB	GBR	United Kingdom	["latitude": 51.51, "longitude": -0.09]	2016-03-20	383	383yNMmFRq0zG
9	red	3	1508	GB	GBR	United Kingdom	["latitude": 53.5, "longitude": -2.19]	2016-03-20	391	391Qn3iLA
10	yellow	5	1191	KR	KOR	Republic of Korea	["latitude": 37.29, "longitude": 127.01]	2016-03-20	455	455L4J9FUG

EXAMPLE 4: STATIC USER-DEFINED SCHEMA

AL also supports static custom schemas just like Spark Structured Streaming. This eliminates the need for dynamic schema-on-read inference scans, which trigger additional Spark jobs and schema versions. The schema can be retrieved as a DDL string or a JSON payload.

DDL

```

1 %scala
2 inferALDf.schema.toDDL

String = alarm_status STRING,battery_level BIGINT,c02_level BIGINT,cca2 STRING,cca3
STRING,cn STRING,coordinates STRUCT<latitude: DOUBLE, longitude: DOUBLE>,date
STRING,device_id BIGINT,device_serial_number STRING,device_type STRING,epoch_time_
milliseconds BIGINT,humidity BIGINT,ip STRING,scale STRING,temp DOUBLE,timestamp
STRING,_rescued_data STRING

```

JSON

```

1 %scala
2 spark.read.json(repoSchemaPath + "/_schemas").select("dataSchemaJson").
3 where("dataSchemaJson is not null").first()

```

```

1 org.apache.spark.sql.Row = [{"type":"struct","fields":[{"name":"alarm_
2 status","type":"string","nullable":true,"metadata":{}},{name":"battery_level","type":
3 "long","nullable":true,"metadata":{}},{name":"c02_level","type":"long","nullable":true,
4 "metadata":{}},{name":"cca2","type":"string","nullable":true,"metadata":{}},{name":
5 "cca3","type":"string","nullable":true,"metadata":{}},{name":"cn","type":"string",
6 "nullable":true,"metadata":{}},{name":"coordinates","type":{"type":"struct","fields":
7 [{"name":"latitude","type":"double","nullable":true,"metadata":{}},{name":"longitude",
8 "type":"double","nullable":true,"metadata":{}}],"nullable":true,"metadata":{}},
9 {"name":"date","type":"string","nullable":true,"metadata":{}},{name":"device_id",
10 "type":"long","nullable":true,"metadata":{}},{name":"device_serial_number","type":
11 "string","nullable":true,"metadata":{}},{name":"device_type","type":"string",
12 "nullable":true,"metadata":{}},{name":"epoch_time_milliseconds","type":"long",
13 "nullable":true,"metadata":{}},{name":"humidity","type":"long","nullable":true,
14 "metadata":{}},{name":"ip","type":"string","nullable":true,"metadata":{}},{name":
15 "scale","type":"string","nullable":true,"metadata":{}},{name":"temp","type":"double",
16 "nullable":true,"metadata":{}},{name":"timestamp","type":"string","nullable":true,
17 "metadata":{}}]}]

```

Here's an example of how to generate a user-defined **StructType (Scala)** | **StructType (Python)** via DDL DataFrame command or JSON queried from AL schema repository.

```

1 %scala
2 import org.apache.spark.sql.types.{DataType, StructType}

3 val ddl = """alarm_status STRING, battery_level BIGINT,c02_level BIGINT,cca2 STRING,
4 cca3 STRING, cn STRING, coordinates STRUCT<latitude: DOUBLE, longitude: DOUBLE>,
5 date STRING, device_id BIGINT, device_serial_number STRING, device_type STRING,
6 epoch_time_milliseconds BIGINT, humidity BIGINT, ip STRING, scale STRING,temp DOUBLE,
7 timestamp STRING, _rescued_data STRING"""

8 val ddlSchema = StructType.fromDDL(ddl)

9 val json = """{"type":"struct","fields":[{"name":"alarm_status","type":"string",
10 "nullable":true,"metadata":{}}, {"name":"battery_level","type":"long","nullable":
11 true,"metadata":{}}, {"name":"c02_level","type":"long","nullable":true, "metadata"
12 :{}}, {"name":"cca2","type":"string","nullable":true,"metadata":{}}, {"name":"cca3",
13 "type":"string","nullable":true,"metadata":{}}, {"name":"cn","type":"string","nullable":
14 true,"metadata":{}}, {"name":"coordinates","type":{"type":"struct","fields":
15 [{"name":"latitude","type":"double","nullable":true,"metadata":{}}, {"name":"longitude",
16 "type":"double","nullable":true,"metadata":{}}], "nullable":true,"metadata":{}}, {"name"
17 : "date","type":"string","nullable":true,"metadata":{}}, {"name":"device_id","type":
18 "long","nullable":true,"metadata":{}}, {"name":"deviceserial_number","type":"string",
19 "nullable":true,"metadata":{}}, {"name":"device_type","type":"string","nullable":true,
20 "metadata":{}}, {"name":"epochtime_milliseconds","type":"long","nullable":true,"metadata":{}},
21 {"name":"humidity","type":"long","nullable":true,"metadata":{}}, {"name":
22 "ip","type":"string","nullable":true,"metadata":{}}, {"name":"scale","type":"string",
23 "nullable":true,"metadata":{}}, {"name":"temp","type":"double","nullable":true,
24 "metadata":{}}, {"name":"timestamp","type":"string","nullable":true,"metadata":{}}]}"""

25 val jsonSchema = DataType.fromJson(json).asInstanceOf[StructType]

```

```

1 %scala
2 val schemaA1Df = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .schema(jsonSchema) // schema structtype definition
6   .load(jsonSchema1Path)
7 )

```

```

1 %scala
2 schemaA1Df.printSchema

```

```

1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 |   |-- latitude: double (nullable = true)
10 |   |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: long (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_milliseconds: long (nullable = true)
16 |-- humidity: long (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)

```

Passing in the schema definition will enforce the stream. AL also provides a schema enforcement option achieving basically the same results as providing a static StructType schema-on-read. This method will be covered in Example 7.

```

1 %scala
2 display(schemaA1Df.limit(10))

```

	alarm_status	battery_level	c02_level	cca2	cca3	cn	coordinates	date	device_id	device_serial_num
1	yellow	3	1082	US	USA	United States	{ "latitude": 38, "longitude": -97 }	2016-03-20	62	62IH8oKr8aiT
2	green	0	920	US	USA	null	{ "latitude": 47, "longitude": 8 }	2016-03-20	241	241un29KmR
3	yellow	7	1004	US	USA	United States	{ "latitude": 42.36, "longitude": -71.05 }	2016-03-20	260	260F7DTEbnz
4	yellow	9	1081	DE	DEU	Germany	{ "latitude": 51.45, "longitude": 7.02 }	2016-03-20	298	298hJceoQv0
5	yellow	8	1311	US	USA	United States	{ "latitude": 38.88, "longitude": -92.4 }	2016-03-20	301	301wHcyNzw
6	red	1	1484	IN	IND	India	{ "latitude": 20, "longitude": 77 }	2016-03-20	334	334DUAg4mbXi
7	yellow	9	1266	US	USA	null	{ "latitude": 47, "longitude": 8 }	2016-03-20	349	349gRZSGtcGR
8	yellow	1	1085	GB	GBR	United Kingdom	{ "latitude": 51.51, "longitude": -0.09 }	2016-03-20	383	383yNMmfrq0zG
9	red	3	1508	GB	GBR	United Kingdom	{ "latitude": 53.5, "longitude": -2.19 }	2016-03-20	391	391Qn3ILA
10	yellow	5	1191	KR	KOR	Republic of Korea	{ "latitude": 37.29, "longitude": 127.01 }	2016-03-20	455	455L4J9FUG

EXAMPLE 5: SCHEMA DRIFT

AL stores new columns and data types via the rescue column. This column captures schema changes-on-read. The stream does not fail when schema and data type mismatches are discovered. This is a very impressive feature!

```
1 %scala
2 val driftAlDf = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .option("cloudFiles.schemaLocation", repoSchemaPath)
6   .option("cloudFiles.inferColumnTypes", true)
7   .option("cloudFiles.schemaEvolutionMode", "rescue") // schema drift tracking
8   .load(rawBasePath + "/*.json")
9 )
```

```
1 %scala
2 driftAlDf.printSchema
```

```
1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 |   |-- latitude: double (nullable = true)
10 |   |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: long (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_milliseconds: long (nullable = true)
16 |-- humidity: long (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)
21 |-- _rescued_data: string (nullable = true)
```

The rescue column preserves schema drift such as newly appended columns and/or different data types via a JSON string payload. This payload can be parsed via Spark DataFrame or Dataset APIs to analyze schema drift scenarios. The source file path for each individual row is also available in the rescue column to investigate the root cause.

```
1 %scala
2 display(driftAlDf.where("_rescued_data is not null").limit(10))
```

seconds	humidity	ip	scale	temp	timestamp	_rescued_data
1	null	213.161.254.1	Fahrenheit	51.8	2016-03-20 03:20:54.119	{\"location\":{\"cca3\":\"NOR\",\"cn\":\"Norway\",\"cca2\":\"NO\"},\"latitude\":62.47,\"device_serial_number_device_type\":\"12h2Pea\",\"sensor-pad\":\"\",\"longitude\":6.15,\"humidity\":70.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
2	null	68.28.91.22	Fahrenheit	53.6	2016-03-20 03:20:54.126	{\"location\":{\"cca3\":\"USA\",\"cn\":\"United States\",\"cca2\":\"US\"},\"latitude\":38.0,\"device_serial_number_device_type\":\"1272Xim0d\",\"sensor-pad\":\"\",\"longitude\":-97.0,\"humidity\":92.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
3	null	59.90.65.1	Fahrenheit	73.4	2016-03-20 03:20:54.133	{\"location\":{\"cca3\":\"IND\",\"cn\":\"India\",\"cca2\":\"IN\"},\"latitude\":12.98,\"device_serial_number_device_type\":\"1230upA\",\"meter-gauge\":\"\",\"longitude\":77.58,\"humidity\":47.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
4	null	213.7.14.1	Fahrenheit	75.2	2016-03-20 03:20:54.141	{\"location\":{\"cca3\":\"CYP\",\"cn\":\"Cyprus\",\"cca2\":\"CY\"},\"latitude\":35.0,\"device_serial_number_device_type\":\"38VQv8fEg\",\"sensor-pad\":\"\",\"longitude\":33.0,\"humidity\":47.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
5	null	62.126.16.74	Fahrenheit	80.6	2016-03-20 03:20:54.149	{\"location\":{\"cca3\":\"DEU\",\"cn\":\"Germany\",\"cca2\":\"DE\"},\"latitude\":49.46,\"device_serial_number_device_type\":\"448DeWGL\",\"sensor-pad\":\"\",\"longitude\":11.1,\"humidity\":63.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
6	null	170.37.224.1	Fahrenheit	77	2016-03-20 03:20:54.152	{\"location\":{\"cca3\":\"USA\",\"cn\":\"United States\",\"cca2\":\"US\"},\"latitude\":42.28,\"device_serial_number_device_type\":\"49Yes0XwI\",\"meter-gauge\":\"\",\"longitude\":-71.44,\"humidity\":70.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
7	null	38.99.198.198	Fahrenheit	53.6	2016-03-20 03:20:54.162	{\"location\":{\"cca3\":\"USA\",\"cn\":\"United States\",\"cca2\":\"US\"},\"latitude\":38.0,\"device_serial_number_device_type\":\"64dcln\",\"sensor-pad\":\"\",\"longitude\":-97.0,\"humidity\":55.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
8	null	218.248.255.30	Fahrenheit	62.6	2016-03-20 03:20:54.169	{\"location\":{\"cca3\":\"IND\",\"cn\":\"India\",\"cca2\":\"IN\"},\"latitude\":12.98,\"device_serial_number_device_type\":\"77KW3VAB55\",\"meter-gauge\":\"\",\"longitude\":77.58,\"humidity\":82.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
9	null	159.128.0.181	Fahrenheit	89.6	2016-03-20 03:20:54.171	{\"location\":{\"cca3\":\"CAN\",\"cn\":\"Canada\",\"cca2\":\"CA\"},\"latitude\":50.01,\"device_serial_number_device_type\":\"80TY4gWSMH\",\"sensor-pad\":\"\",\"longitude\":-97.22,\"humidity\":57.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}
10	null	24.32.20.1	Fahrenheit	82.4	2016-03-20 03:20:54.179	{\"location\":{\"cca3\":\"USA\",\"cn\":\"United States\",\"cca2\":\"US\"},\"latitude\":39.33,\"device_serial_number_device_type\":\"94HL9ChD\",\"sensor-pad\":\"\",\"longitude\":-120.25,\"humidity\":66.0,\"_file_path\":\"dbfs:/user/garrett.peternel@databricks.com/raw/iot-schema-2.json/iot_schema_2.json\"}

EXAMPLE 6: SCHEMA EVOLUTION

AL merges schemas as new columns arrive via schema evolution mode. New schema JSON will be updated and stored as a new version in the specified schema repository location.

```
%scala
val evolveAlDf = (spark
  .readStream.format("cloudfiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", repoSchemaPath)
  .option("cloudFiles.inferColumnTypes", true)
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns") // schema evolution
  .load(rawBasePath + "/*.json")
)
```

```
%scala
evolveAlDf.printSchema // original schema
```

```
root
|-- alarm_status: string (nullable = true)
|-- battery_level: long (nullable = true)
|-- c02_level: long (nullable = true)
|-- cca2: string (nullable = true)
|-- cca3: string (nullable = true)
|-- cn: string (nullable = true)
|-- coordinates: struct (nullable = true)
| |-- latitude: double (nullable = true)
| |-- longitude: double (nullable = true)
|-- date: string (nullable = true)
|-- device_id: long (nullable = true)
|-- device_serial_number: string (nullable = true)
|-- device_type: string (nullable = true)
|-- epoch_time_milliseconds: long (nullable = true)
|-- humidity: long (nullable = true)
|-- ip: string (nullable = true)
|-- scale: string (nullable = true)
|-- temp: double (nullable = true)
|-- timestamp: string (nullable = true)
|-- _rescued_data: string (nullable = true)
```

```
%scala
display(evolveAlDf.limit(10)) // # stream will fail
```

AL purposely fails the stream with an `UnknownFieldException` error when it detects a schema change via dynamic schema inference. The updated schema instance is created as a new version and metadata file in the schema repository location, and will be used against the input data after restarting the stream.


```

1 %scala
2 val evolveAldf = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .option("cloudFiles.schemaLocation", repoSchemaPath)
6   .option("cloudFiles.inferColumnTypes", true)
7   .option("cloudFiles.schemaHints", "humidity DOUBLE")
8   .option("cloudFiles.schemaEvolutionMode", "addNewColumns") // schema evolution
9   .load(rawBasePath + "/*.json")
10  )

```

```

1 %scala
2 evolveAldf.printSchema // evolved schema

```

```

1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 |   |-- latitude: double (nullable = true)
10 |   |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: long (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_milliseconds: long (nullable = true)
16 |-- humidity: double (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)
21 |-- device_serial_number_device_type: string (nullable = true)
22 |-- latitude: double (nullable = true)
23 |-- location: struct (nullable = true)
24 |   |-- cca2: string (nullable = true)
25 |   |-- cca3: string (nullable = true)
26 |   |-- cn: string (nullable = true)
27 |-- longitude: double (nullable = true)
28 |-- _rescued_data: string (nullable = true)

```

AL has evolved the schema to merge the newly acquired data fields.

```

1 %scala
2 display(evolveAldf.where("device_serial_number_device_type is not null").limit(10))

```

	temp	timestamp	device_serial_number_device_type	latitude	location	longitude	_rescued_data
1	51.8	2016-03-20 03:20:54.119	["2n2Pea","sensor-pad"]	62.47	["cca2": "NO", "cca3": "NOR", "cn": "Norway"]	6.15	null
2	53.6	2016-03-20 03:20:54.126	["12Y2klm0q","sensor-pad"]	38	["cca2": "US", "cca3": "USA", "cn": "United States"]	-97	null
3	73.4	2016-03-20 03:20:54.133	["230lupA","meter-gauge"]	12.98	["cca2": "IN", "cca3": "IND", "cn": "India"]	77.58	null
4	75.2	2016-03-20 03:20:54.141	["36VQv8fnEg","sensor-pad"]	35	["cca2": "CY", "cca3": "CYP", "cn": "Cyprus"]	33	null
5	80.6	2016-03-20 03:20:54.149	["448DeWGL","sensor-pad"]	49.46	["cca2": "DE", "cca3": "DEU", "cn": "Germany"]	11.1	null
6	77	2016-03-20 03:20:54.152	["49YesGXwt","meter-gauge"]	42.28	["cca2": "US", "cca3": "USA", "cn": "United States"]	-71.44	null
7	53.6	2016-03-20 03:20:54.162	["64djcln","sensor-pad"]	38	["cca2": "US", "cca3": "USA", "cn": "United States"]	-97	null
8	62.6	2016-03-20 03:20:54.169	["77IKW3YAB55","meter-gauge"]	12.98	["cca2": "IN", "cca3": "IND", "cn": "India"]	77.58	null
9	89.6	2016-03-20 03:20:54.171	["80TY4dWSMH","sensor-pad"]	50.01	["cca2": "CA", "cca3": "CAN", "cn": "Canada"]	-97.22	null
10	82.4	2016-03-20 03:20:54.179	["94HL9jChD","sensor-pad"]	39.33	["cca2": "US", "cca3": "USA", "cn": "United States"]	-120.25	null

The newly merged schema transformed by AL is stored in the original schema repository path as version 1 along with the base version 0 schema. This history is valuable for tracking changes to schema over time, as well as quickly retrieving DDL on the fly for schema enforcement.

Schema Repository

```
1 %scala
2 display(dbutils.fs.ls(repoSchemaPath + "/_schemas"))
```

	path	name	size	modificationTime
1	dbfs:/user/garrett.peternel@databricks.com/repo/ot-ddl.json/_schemas/0	0	1628	1709325391000
2	dbfs:/user/garrett.peternel@databricks.com/repo/ot-ddl.json/_schemas/1	1	2209	1709325578000

Schema Metadata

```
1 %scala
2 display(spark.read.json(repoSchemaPath + "/_schemas"))
```

	_corrupt_record	dataSchemaJson	partitionSchemaJson
1	v1	null	null
2	null	{ "type": "struct", "fields": [{ "name": "alarm_status", "type": "string", "nullable": true, "metadata": {} }, { "name": "battery_level", "type": "long", "nullable": true, "metadata": {} }, { "name": "c02_level", "type": "long", "nullable": true, "metadata": {} }, { "name": "cca2", "type": "string", "nullable": true, "metadata": {} }, { "name": "cca3", "type": "string", "nullable": true, "metadata": {} }, { "name": "cn", "type": "string", "nullable": true, "metadata": {} }, { "name": "coordinates", "type": "type": "struct", "fields": [{ "name": "latitude", "type": "do...	{ "type": "struct", "fields": [] }
3	v1	null	null
4	null	{ "type": "struct", "fields": [{ "name": "alarm_status", "type": "string", "nullable": true, "metadata": {} }, { "name": "battery_level", "type": "long", "nullable": true, "metadata": {} }, { "name": "c02_level", "type": "long", "nullable": true, "metadata": {} }, { "name": "cca2", "type": "string", "nullable": true, "metadata": {} }, { "name": "cca3", "type": "string", "nullable": true, "metadata": {} }, { "name": "cn", "type": "string", "nullable": true, "metadata": {} }, { "name": "coordinates", "type": "type": "struct", "fields": [{ "name": "latitude", "type": "do...	{ "type": "struct", "fields": [] }

Schema evolution can be a messy problem if frequent. With AL and Delta Lake it becomes easier and simpler to manage. Adding new columns is relatively straightforward as AL combined with Delta Lake uses schema evolution to append them to the existing schema. Note: the values for these columns will be NULL for data already processed. The greater challenge occurs when the data types change because there will be a type mismatch against the data already processed. Currently, the "safest" approach is to perform a complete overwrite of the target Delta table to refresh all data with the changed data type(s). Depending on the data volume this operation is also relatively straightforward if infrequent. However, if data types are changing daily/weekly then this operation is going to be very costly to reprocess large data volumes. This can be an indication that the business needs to improve their data strategy.

Constantly changing schemas can be a sign of a weak data governance strategy and lack of communication with the data business owners. Ideally, organizations should have some kind of SLA for data acquisition and know the expected schema. Raw data stored in the landing zone should also follow some kind of pre-ETL strategy (e.g., ontology, taxonomy, partitioning) for better incremental loading performance into the lakehouse. Skipping these steps can cause a plethora of data management issues that will negatively impact downstream consumers building data analytics, BI, and AI/ML pipelines and applications. If upstream schema and formatting issues are never addressed, downstream pipelines will consistently break and result in increased cloud storage and compute costs. Garbage in, garbage out.

EXAMPLE 7: SCHEMA ENFORCEMENT

AL validates data against the linked schema version stored in repository location via schema enforcement mode. Schema enforcement is a schema-on-write operation, and only ingested data matching the target Delta Lake schema will be written to output. Any future input schema changes will be ignored, and AL streams will continue working without failure. Schema enforcement is a very powerful feature of AL and Delta Lake. It ensures only clean and trusted data will be inserted into downstream Silver/Gold datasets used for data analytics, BI, and AI/ML pipelines and applications.

```
1 %scala
2 val enforceAlDf = (spark
3   .readStream.format("cloudfiles")
4   .option("cloudFiles.format", "json")
5   .option("cloudFiles.schemaLocation", repoSchemaPath)
6   .option("cloudFiles.schemaEvolutionMode", "none") // schema enforcement
7   .schema(jsonSchema)
8   .load(rawBasePath + "/*.json")
9 )
```

```
1 %scala
2 enforceAlDf.printSchema
```

```
1 root
2 |-- alarm_status: string (nullable = true)
3 |-- battery_level: long (nullable = true)
4 |-- c02_level: long (nullable = true)
5 |-- cca2: string (nullable = true)
6 |-- cca3: string (nullable = true)
7 |-- cn: string (nullable = true)
8 |-- coordinates: struct (nullable = true)
9 | |-- latitude: double (nullable = true)
10 | |-- longitude: double (nullable = true)
11 |-- date: string (nullable = true)
12 |-- device_id: long (nullable = true)
13 |-- device_serial_number: string (nullable = true)
14 |-- device_type: string (nullable = true)
15 |-- epoch_time_milliseconds: long (nullable = true)
16 |-- humidity: long (nullable = true)
17 |-- ip: string (nullable = true)
18 |-- scale: string (nullable = true)
19 |-- temp: double (nullable = true)
20 |-- timestamp: string (nullable = true)
```

Please note the rescue column is no longer available in this example because schema enforcement has been enabled. However, a rescue column can still be configured separately as an AL option if desired. In addition, schema enforcement mode uses the latest schema version in the repository to enforce incoming data. For older versions, set a user-defined schema as explained in Example 4.

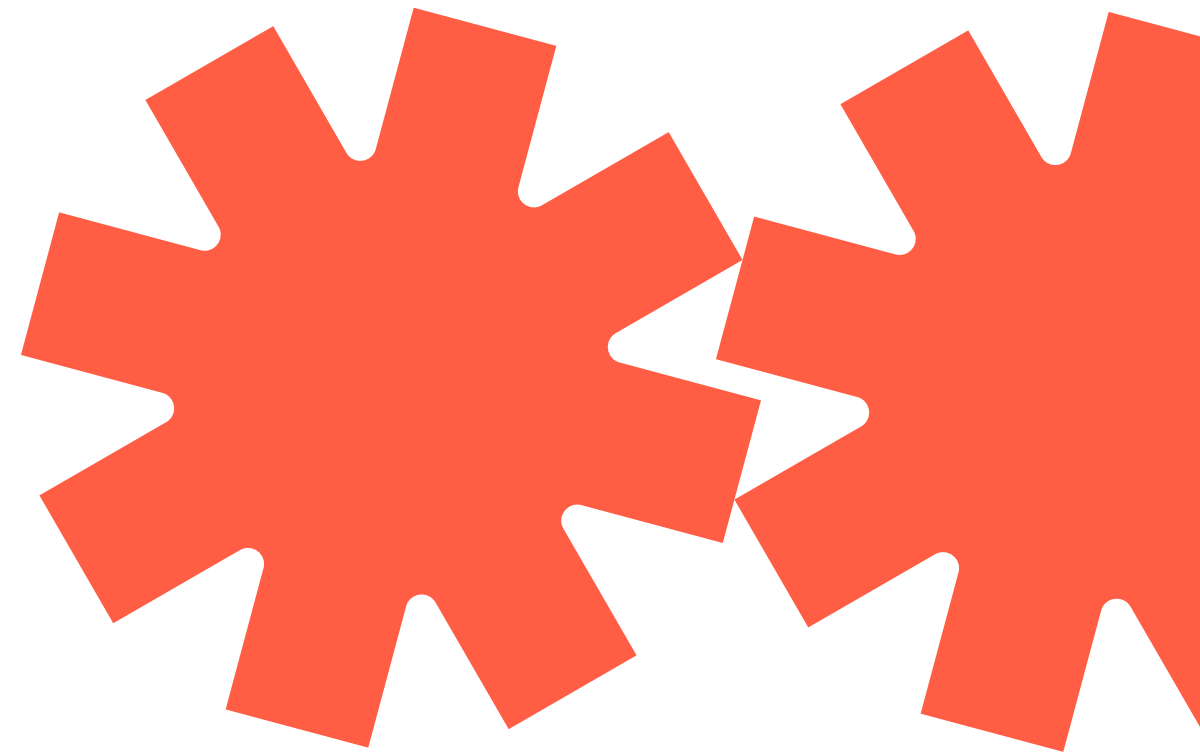
```
1 %scala
2 display(enforceAlDf.limit(10))
```

	alarm_status	battery_level	c02_level	cca2	cca3	cn	coordinates	date	device_id	device_serial_num
1	yellow	3	1082	US	USA	United States	["latitude": 38, "longitude": -97]	2016-03-20	62	62fH8oKrBaiT
2	green	0	920	US	USA	null	["latitude": 47, "longitude": 8]	2016-03-20	241	241un29KmR
3	yellow	7	1004	US	USA	United States	["latitude": 42.36, "longitude": -71.05]	2016-03-20	260	260F7DTebnz
4	yellow	9	1081	DE	DEU	Germany	["latitude": 51.45, "longitude": 7.02]	2016-03-20	298	298hJceoQyD
5	yellow	8	1311	US	USA	United States	["latitude": 38.88, "longitude": -92.4]	2016-03-20	301	301wHcyNzw
6	red	1	1484	IN	IND	India	["latitude": 20, "longitude": 77]	2016-03-20	334	334DUAg4mbXi
7	yellow	9	1266	US	USA	null	["latitude": 47, "longitude": 8]	2016-03-20	349	349gRZSGtcGR
8	yellow	1	1085	GB	GBR	United Kingdom	["latitude": 51.51, "longitude": -0.09]	2016-03-20	383	383yNMmFRqDzG
9	red	3	1508	GB	GBR	United Kingdom	["latitude": 53.5, "longitude": -2.19]	2016-03-20	391	391Qn3ILA
10	yellow	5	1191	KR	KOR	Republic of Korea	["latitude": 37.29, "longitude": 127.01]	2016-03-20	455	455L4J9FUG

CONCLUSION

At the end of the day, data issues are inevitable. However, the key is to limit data pollution as much as possible and have methods to detect discrepancies, changes and history via schema management. Databricks Auto Loader provides many solutions for schema management, as illustrated by the examples in this chapter. Having a solidified data governance and landing zone strategy will make ingestion and streaming easier and more efficient for loading data into the lakehouse. Whether it is simply converting raw JSON data incrementally to the Bronze layer as Delta Lake format, or having a repository to store schema metadata, AL makes your job easier. It acts as an anchor to building a resilient lakehouse architecture that provides reusable, consistent, reliable and performant data throughout the data and AI lifecycle.

HTML notebooks (Spark Scala and Spark Python) with code and both sample datasets can be found at the GitHub repo [here](#).



From Idea to Code: Building With the Databricks SDK for Python

by Kimberly Mahoney

The focus of this chapter is to demystify the **Databricks SDK for Python** — the authentication process and the components of the SDK — by walking through the start-to-end development process. I'll also be showing how to utilize IntelliSense and the debugger for real-time suggestions in order to reduce the amount of context-switching from the IDE to documentation and code examples.

What is the Databricks SDK for Python . . . and why you should use it

The Databricks Python SDK lets you interact with the Databricks Platform programmatically using Python. It covers the entire Databricks API surface and Databricks REST operations. While you can interact directly with the API via curl or a library like 'requests' there are benefits to utilizing the SDKs such as:

- Secure and simplified authentication via **Databricks client-unified authentication**
- Built-in debug logging with sensitive information automatically redacted
- Support to wait for long-running operations to finish (kicking off a job, starting a cluster)
- Standard iterators for paginated APIs (we have multiple pagination types in our APIs!)
- Retrying on transient errors

There are numerous practical applications, such as building multi-tenant web applications that interact with your ML models or a robust UC migration toolkit like Databricks Labs project **UCX**. Don't forget the silent workhorses — those simple utility scripts that are more limited in scope but automate an annoying task such as bulk updating cluster policies, dynamically adding users to groups or simply writing data files to UC Volumes. Implementing these types of scripts is a great way to familiarize yourself with the Python SDK and Databricks APIs.

SCENARIO

Imagine my business is establishing best practices for development and CI/CD on Databricks. We're adopting **DABs** to help us define and deploy workflows in our development and production workspaces, but in the meantime, we need to audit and clean up our current environments. We have a lot of jobs people created in our dev workspace via the UI. One of the platform admins observed many of these jobs are inadvertently configured to run on a recurring schedule, racking up unintended costs. As part of the cleanup process, we want to identify any scheduled jobs in our development workspace with an option to pause them. We'll need to figure out:

- How to install the SDK
- How to connect to the Databricks workspace
- How to list all the jobs and examine their attributes
- How to log the problematic jobs — or a step further, how to call the API to pause their schedule

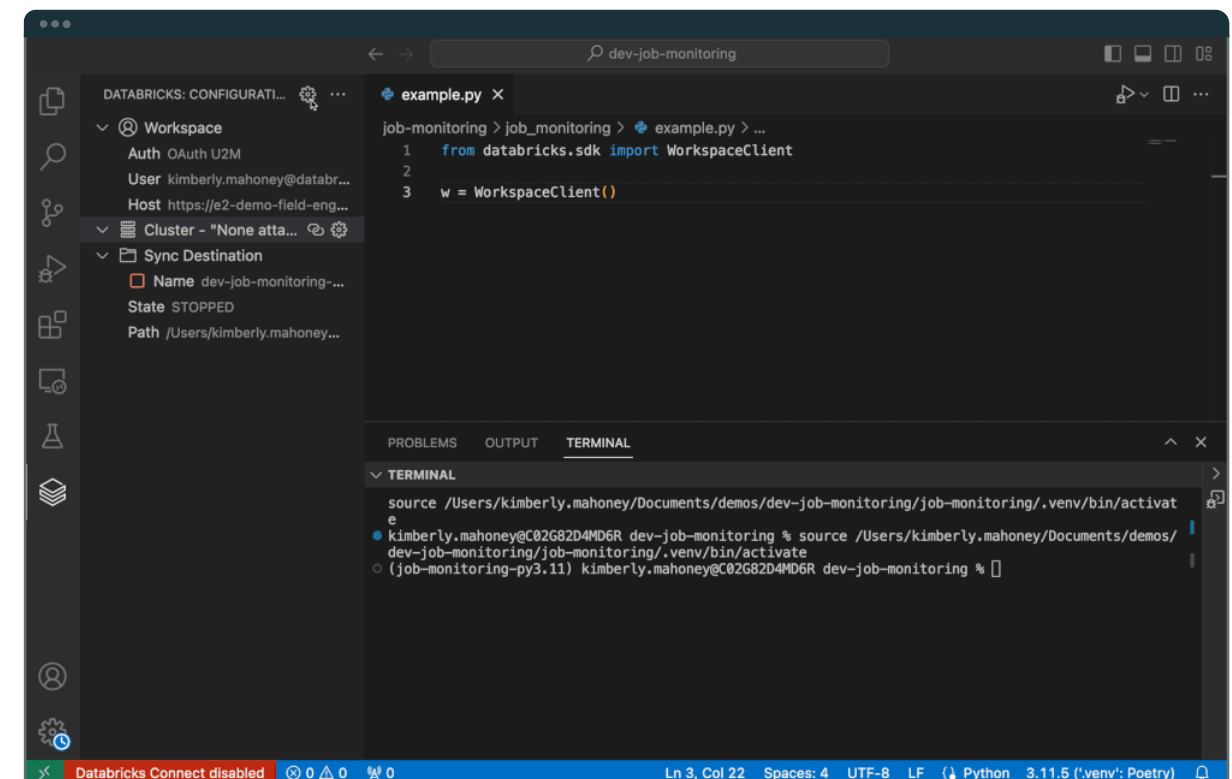
DEVELOPMENT ENVIRONMENT

Before diving into the code, you need to set up your development environment. I highly recommend using an IDE that has a comprehensive code completion feature as well as a debugger. Code completion features, such as **IntelliSense in VS Code**, are really helpful when learning new libraries or APIs — they provide useful contextual information, autocompletion, and aid in code navigation. For this chapter, I'll be using Visual Studio Code so I can also make use of the **Databricks Extension** as well as **Pylance**. You'll also need to install the **databricks-sdk** (**docs**). In this chapter, I'm using Poetry + Pyenv. The setup is similar for other tools — just 'poetry add databricks-sdk' or alternatively 'pip install databricks-sdk' in your environment.

AUTHENTICATION

The next step is to authorize access to Databricks so we can work with our workspace. There are several ways to do this, but because I'm using the VS Code Extension, I'll take advantage of its authentication integration. It's one of the tools that uses **unified client authentication** — that just means all these development tools follow the same process and standards for authentication and if you set up auth for one, you can reuse it among the other tools. I set up both the CLI and VS Code Extension previously, but here is a primer on **setting up the CLI** and **installing the extension**. Once you've connected successfully, you'll see a notification banner in the lower right-hand corner and see two hidden files generated in the **.databricks** folder — **project.json** and **databricks.env** (don't worry, the extension also handles adding these to **.gitignore**).

For this example, while we're interactively developing in our IDE, we'll be using what's called U2M (user-to-machine) OAuth. We won't get into the technical details, but OAuth is a secure protocol that handles authorization to resources without passing sensitive user credentials such as PAT or username/password that persist much longer than the one-hour short-lived OAuth token.



OAuth flow for the Databricks Python SDK

WORKSPACECLIENT VS. ACCOUNTCLIENT

The Databricks API is split into two primary categories — account and workspace. They let you manage different parts of Databricks, like user access at the account level or cluster policies in a workspace. The SDK reflects this with two clients that act as our entry points to the SDK — the `WorkspaceClient` and `AccountClient`. For our example we'll be working at the workspace level, so I'll be initializing the `WorkspaceClient`. If you're unsure which client to use, check out the [SDK documentation](#).

Tip: Because we ran the previous steps to authorize access via unified client auth, the SDK will automatically use the necessary Databricks environment variables, so there's no need for extra configurations when setting up your client. All we need are these two lines of code:

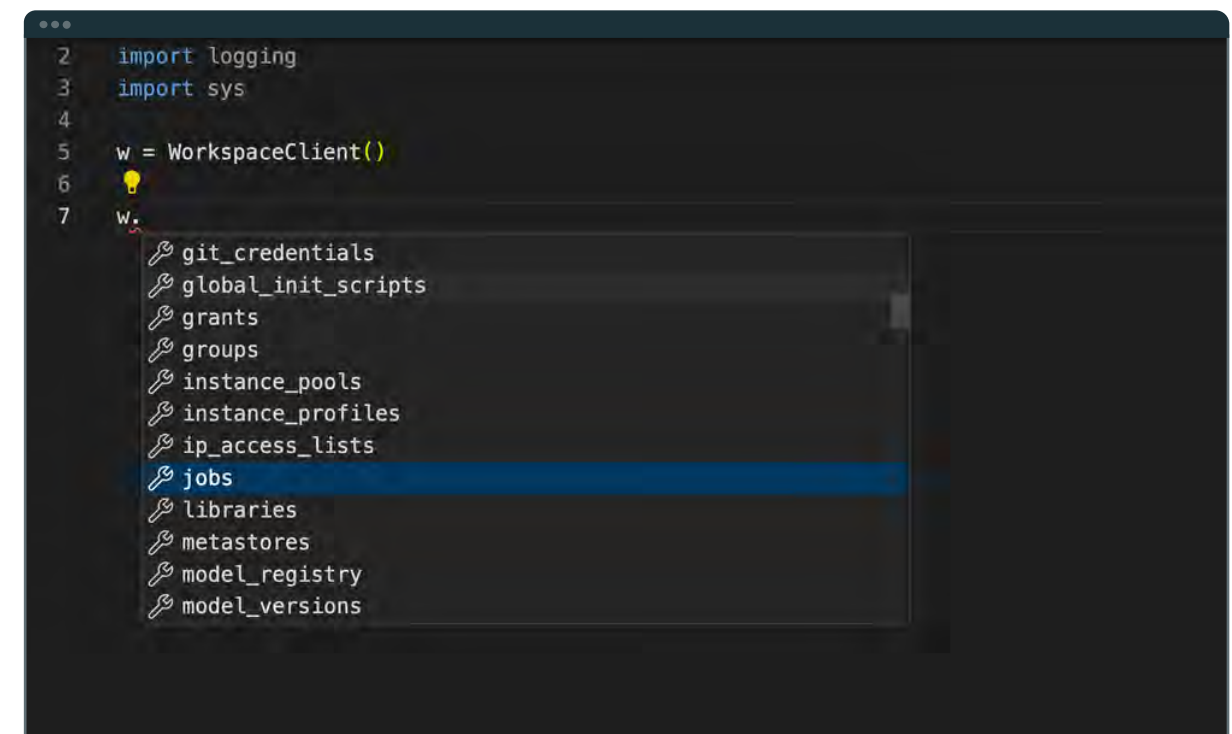
■ Initializing our `WorkspaceClient`

```
1 from databricks.sdk import WorkspaceClient
2 w = WorkspaceClient()
```

MAKING API CALLS AND INTERACTING WITH DATA

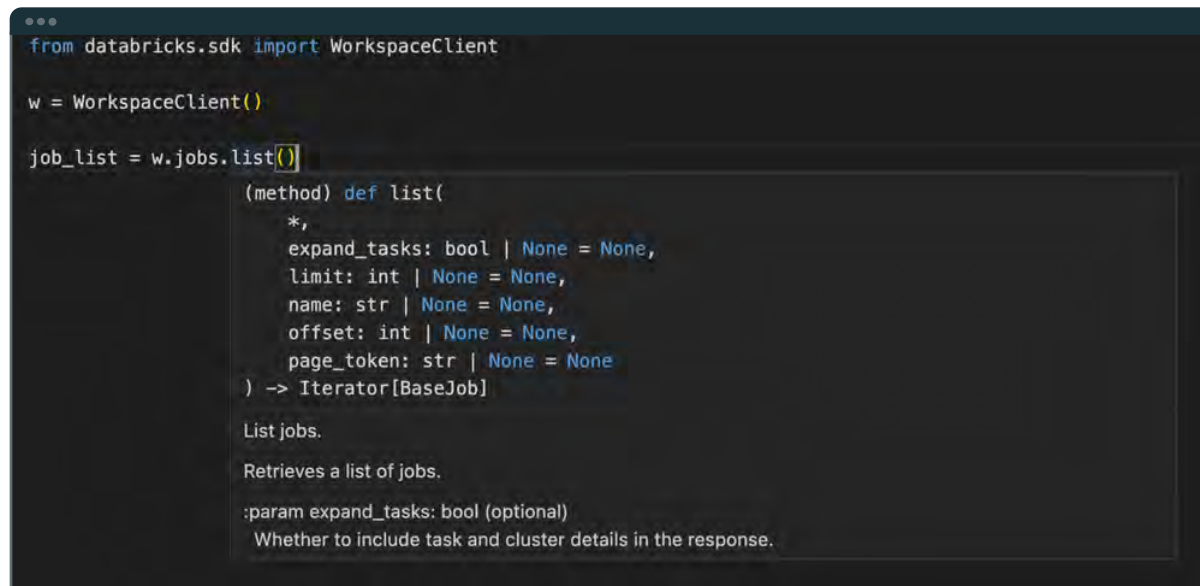
The `WorkspaceClient` we instantiated will allow us to interact with different APIs across the Databricks workspace services. A **service** is a smaller component of the Databricks Platform — e.g., Jobs, Compute, Model Registry.

In our example, we'll need to call the Jobs API in order to retrieve a list of all the jobs in the workspace.



Services accessible via the Python SDK

This is where IntelliSense really comes in handy. Instead of context switching between the IDE and the Documentation page, I can use autocomplete to provide a list of methods as well as examine the method description, the parameters and return types from within the IDE. I know the first step is getting a list of all the jobs in the workspace:



```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

job_list = w.jobs.list()
```

(method) def list(*, expand_tasks: bool | None = None, limit: int | None = None, name: str | None = None, offset: int | None = None, page_token: str | None = None) -> Iterator[BaseJob]

List jobs.

Retrieves a list of jobs.

:param expand_tasks: bool (optional)
Whether to include task and cluster details in the response.

As you can see, it returns an iterator over an object called BaseJob. Before we talk about what a BaseJob actually is, it'll be helpful to understand how data is used in the SDK. To interact with data you are sending to and receiving from the API, the Python SDK takes advantage of Python **data classes** and **enums**. The main advantage of this approach over passing around dictionaries is improved readability while also minimizing errors through enforced type checks and validations.

You can construct objects with data classes and interact with enums.

For example:

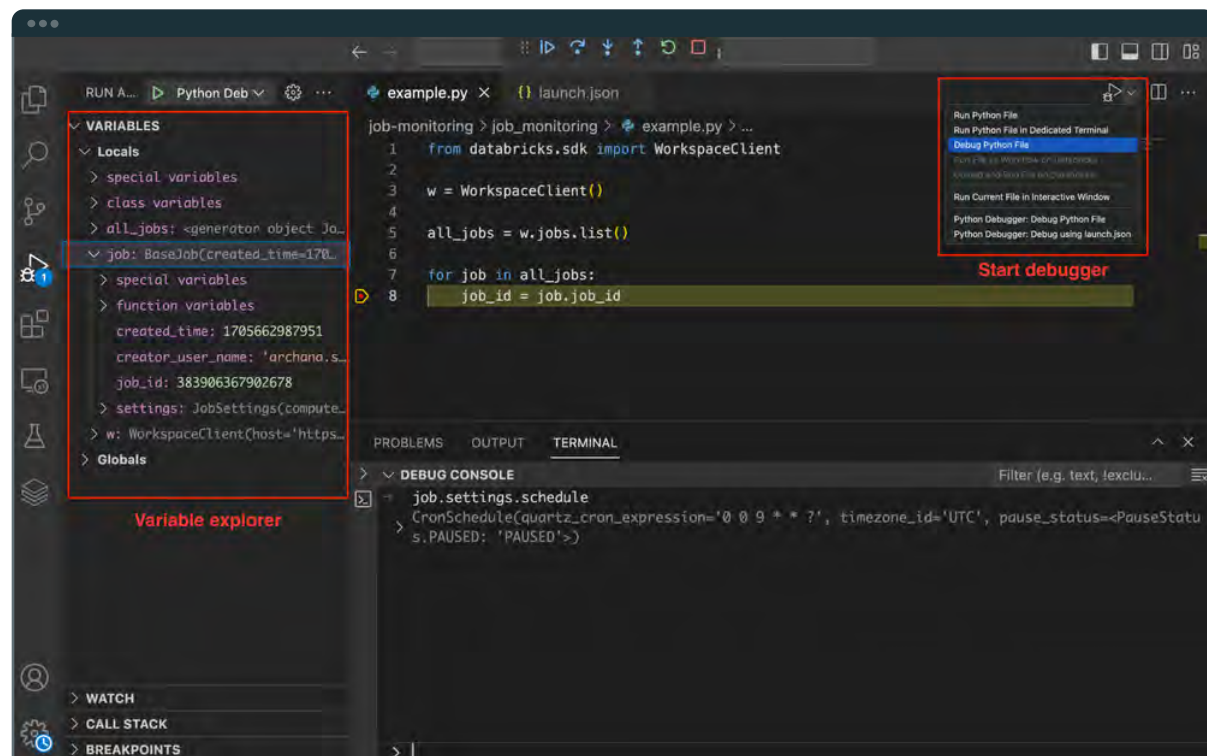
- *Creating an Employee via Employee DataClass and company departments, using enums for possible department values*



```
1 from dataclasses import dataclass
2 from enum import Enum
3 from typing import Optional
4
5 class CompanyDepartment(Enum):
6     MARKETING = 'MARKETING'
7     SALES = 'SALES'
8     ENGINEERING = 'ENGINEERING'
9
10 @dataclass
11 class Employee:
12     name: str
13     email: str
14     department: Optional[CompanyDepartment] = None
15
16 emp = Employee('Bob', 'bob@company.com', CompanyDepartment.ENGINEERING)
```

In the Python SDK all of the data classes, enums and APIs belong to the same module for a service located under **databricks.sdk.service** — e.g., **databricks.sdk.service.jobs**, **databricks.sdk.service.billing**, **databricks.sdk.service.sql**.

For our example, we'll need to loop through all of the jobs and make a decision on whether or not they should be paused. I'll be using a **debugger** in order to look at a few example jobs and get a better understanding of what a 'BaseJob' looks like. The Databricks VS Code extension comes with a debugger that can be used to troubleshoot code issues interactively on Databricks via **Databricks Connect**. But, because I do not need to run my code on a cluster, I'll just be using the standard **Python debugger**. I'll set a breakpoint inside for my loop and use the VS Code Debugger to examine a few examples. A breakpoint allows us to stop code execution and interact with variables during our debugging session. This is preferable over print statements, as you can use the debugging console to interact with the data as well as progress the loop. In this example I'm looking at the **settings** field and drilling down further in the debugging console to take a look at what an example job schedule looks like:



Inspecting BaseJob in the VS Code debugger

We can see a BaseJob has a few top-level attributes and has a more complex Settings type that contains most of the information we care about. At this point, we have our WorkspaceClient and are iterating over the jobs in our workspace. To flag problematic jobs and potentially take some action, we'll need to better understand **job.settings.schedule**. We need to figure out how to programmatically identify if a job has a schedule and flag if it's not paused. For this we'll be using another handy utility for code navigation — **Go to Definition**. I've opted to Open Definition to the Side (⌘ F12) in order to reduce switching to a new window. This will allow us to quickly navigate through the data class definitions without having to switch to a new window or exit our IDE:

As we can see, a BaseJob contains some top-level fields that are common among jobs such as 'job_id' or 'created_time'. A job can also have various settings (JobSettings). These configurations often differ between jobs and encompass aspects like notification settings, tasks, tags and the schedule. We'll be focusing on the schedule field, which is represented by the CronSchedule data class. CronSchedule contains information about the pause status (PauseStatus) of a job. PauseStatus in the SDK is represented as an enum with two possible values — **PAUSED** and **UNPAUSED**.

Tip: VSCode + Pylance provides code suggestions, and you can enable auto imports in your **User Settings or on a per-project basis in Workspace Settings**. By default, only top-level symbols are suggested for auto import and suggested code (see [original GitHub issue](#)). However, the SDK has nested elements we want to generate suggestions for. We actually need to go down 5 levels — databricks.sdk.service.jobs.<Enum|Dataclass>. In order to take full advantage of these features for the SDK, I added a couple of workspace settings:

- *Selection of the VSCode Workspace settings.json*

```

1  ...
2  "python.analysis.autoImportCompletions": true,
3  "python.analysis.indexing": true,
4  "python.analysis.packageIndexDepths": [
5  {
6      "name": "databricks",
7      "depth": 5,
8      "includeAllSymbols": true
9  }
10 ]
11 ...

```

Putting it all together:

I broke out the policy logic into its own function for unit testing, added some logging and expanded the example to check for any jobs tagged as an exception to our policy. Now we have:

- *Logging out of policy jobs*

```

1  import logging
2
3  from databricks.sdk import WorkspaceClient
4  from databricks.sdk.service.jobs import CronSchedule, JobSettings, PauseStatus
5
6  # Initialize WorkspaceClient
7  w = WorkspaceClient()
8
9  def update_new_settings(job_id, quartz_cron_expression, timezone_id):
10     """Update out of policy job schedules to be paused"""
11     new_schedule = CronSchedule(
12         quartz_cron_expression=quartz_cron_expression,
13         timezone_id=timezone_id,
14         pause_status=PauseStatus.PAUSED,
15     )
16     new_settings = JobSettings(schedule=new_schedule)
17
18     logging.info(f"Job id: {job_id}, new_settings: {new_settings}")
19     w.jobs.update(job_id, new_settings=new_settings)
20
21 def out_of_policy(job_settings: JobSettings):
22     """Check if a job is out of policy.
23     If it unpaused and has a schedule and is not tagged as keep_alive
24     Return true if out of policy, false if in policy
25     """
26
27     tagged = bool(job_settings.tags)
28     proper_tags = tagged and "keep_alive" in job_settings.tags
29     paused = job_settings.schedule.pause_status is PauseStatus.PAUSED
30
31     return not paused and not proper_tags
32
33 all_jobs = w.jobs.list()
34 for job in all_jobs:
35     job_id = job.job_id
36     if job.settings.schedule and out_of_policy(job.settings):
37         schedule = job.settings.schedule
38
39         logging.info(
40             f"Job name: {job.settings.name}, Job id: {job_id}, creator: {job.creator_
41             user_name}, schedule: {schedule}"
42         )
43     ....

```

Now we have not only a working example but also a great foundation for building out a generalized job monitoring tool. We're successfully connecting to our workspace, listing all the jobs and analyzing their settings, and, when we're ready, we can simply call our `update\_new\_settings` function to apply the new paused schedule. It's fairly straightforward to expand this to meet other requirements you may want to set for a workspace — for example, swap job owners to service principles, add tags, edit notifications or audit job permissions. See the example in the [GitHub repository](#).

SCHEDULING A JOB ON DATABRICKS

You can run your script anywhere, but you may want to schedule scripts that use the SDK to run as a Databricks Workflow or job on a small single-node cluster. When running a Python notebook interactively or via automated workflow, you can take advantage of default Databricks Notebook authentication. If you're working with the Databricks WorkspaceClient and your cluster meets the [requirements listed in the docs](#), you can initialize your WorkspaceClient without needing to specify any other configuration options or environment variables — it works automatically out of the box.

The screenshot displays the Databricks job monitoring interface. The main panel shows the output of a Python script, which lists various jobs and their settings. The script output includes job names, IDs, and schedules. The right-hand sidebar provides detailed information about the selected job run, including its ID, status, and duration.

Job Monitoring Output:

```
proper_tags = tagged and keep_alive in job_settings.tags
paused = job_settings.schedule.pause_status is PauseStatus.PAUSED

return not paused and not proper_tags

all_jobs = w.jobs.list()
for job in all_jobs:
    job_id = job.job_id
    if job.settings.schedule and out_of_policy(job.settings):
        schedule = job.settings.schedule

    print(f"Job name: {job.settings.name}, Job id: {job_id}, schedule: {schedule.quartz_cron_expression}")

Job name: test-mlops-project-natya-model-training-job, Job id: 51265477673878, schedule: 0 0 9 * * ?
Job name: test-mlops-project-natya-write-feature-table-job, Job id: 581617178734662, schedule: 0 0 7 * * ?
Job name: test-mlops-project-natya-batch-inference-job, Job id: 661538788172146, schedule: 0 0 11 * * ?
Job name: staging-mlops-project-natya-write-feature-table-job, Job id: 988297457859299, schedule: 0 0 7 * * ?
Job name: staging-mlops-project-natya-model-training-job, Job id: 574748586272652, schedule: 0 0 9 * * ?
Job name: staging-mlops-project-natya-batch-inference-job, Job id: 848787196788542, schedule: 0 0 11 * * ?
Job name: test-mlops-project-stijn-batch-inference-job, Job id: 3381319258929, schedule: 0 0 11 * * ?
Job name: test-mlops-project-stijn-write-feature-table-job, Job id: 182963295384967, schedule: 0 0 7 * * ?
Job name: test-mlops-project-stijn-model-training-job, Job id: 315725851631785, schedule: 0 0 9 * * ?
Job name: staging-mlops-project-stijn-batch-inference-job, Job id: 613191886293224, schedule: 0 0 11 * * ?
Job name: staging-mlops-project-stijn-model-training-job, Job id: 572588951767851, schedule: 0 0 9 * * ?
Job name: staging-mlops-project-stijn-write-feature-table-job, Job id: 563143323231114, schedule: 0 0 7 * * ?
Job name: SAT Driver Notebook, Job id: 741617137828625, schedule: 0 0 8 7 * Mon,Wed,Fri
Job name: Multi_Hop, Job id: 1868544822884933, schedule: 18 57 11 * * ?
```

Task run details:

- Job ID: 64011777346387
- Job run ID: 399648124376431
- Task run ID: 877518783773277
- Run as: kmahoney-sdk
- Launched: Manually
- Started: 01/22/2024, 09:55:33 PM
- Ended: 01/22/2024, 09:58:18 PM
- Duration: 2m 45s
- Queue duration: -
- Status: Succeeded
- Lineage: No lineage information for this job. [Learn more](#)

Notebook: [Users/kimberly.mahoney@databricks.com/job_monitoring](#)

Compute: Job_cluster

Driver: i3.xlarge · Workers: i3.xlarge · 1 worker · On-demand and Spot · fall back to On-demand · 13.3 LTS Photon (includes Apache Spark 3.4.1, Scala 2.12) · us-east-1b

[View details](#) [Spark UI](#) [Logs](#) [Metrics](#)

Dependent libraries: databricks-sdk==0.17.0 (PyPI)

CONCLUSION

In conclusion, the Databricks SDKs offer limitless potential for a variety of applications. We saw how the Databricks SDK for Python can be used to automate a simple yet crucial maintenance task and also saw an example of an OSS project that uses the Python SDK to integrate with the Databricks Platform. Regardless of the application you want to build, the SDKs streamline development for the Databricks Platform and allow you to focus on your particular use case. The key to quickly mastering a new SDK such as the Databricks Python SDK is setting up a proper development environment. Developing in an IDE allows you to take advantage of features such as a debugger, parameter info and code completion, so you can quickly navigate and familiarize yourself with the codebase. Visual Studio Code is a great choice for this as it provides the above capabilities and you can utilize the VSCode extension for Databricks to benefit from unified authentication.

Any feedback is greatly appreciated and welcome. Please raise any issues in the [Python SDK GitHub repository](#). Happy developing!

ADDITIONAL RESOURCES

- [Databricks SDK for Python Documentation](#)
- DAIS Presentation: [Unlocking the Power of Databricks SDKs](#)
- How to install Python libraries in your local development environment: [How to Create and Use Virtual Environments in Python With Poetry](#)
- [Installing the Databricks Extension for Visual Studio Code](#)

03

Ready-to-Use Notebooks and Datasets

This section includes several Solution Accelerators — free, ready-to-use examples of data solutions from different industries ranging from retail to manufacturing and healthcare. Each of the following scenarios includes notebooks with code and step-by-step instructions to help you get started. Get hands-on experience with the Databricks Data Intelligence Platform by trying the following for yourself:



Overall Equipment Effectiveness

Ingest equipment sensor data for metric generation and data-driven decision-making

 [Explore the Solution](#) 



Real-Time Point-of-Sale Analytics

Calculate current inventories for various products across multiple store locations with Delta Live Tables

 [Explore the Solution](#) 



Digital Twins

Leverage digital twins — virtual representations of devices and objects — to optimize operations and gain insights

 [Explore the Solution](#) 



Recommendation Engines for Personalization

Improve customers' user experience and conversion with personalized recommendations

 [Explore the Solution](#) 



Understanding Price Transparency Data

Efficiently ingest large healthcare datasets to create price transparency for better understanding of healthcare costs

 [Explore the Solution](#) 

Additional Solution Accelerators with ready-to-use notebooks can be found here:

04

Case Studies

**INDUSTRY****Automotive****SOLUTION****Data-Driven ESG, Customer Entity Resolution, Demand Forecasting, Product Matching****PLATFORM****Workflows, Unity Catalog, Delta Sharing, ETL****CLOUD****Azure**

Cox Automotive — changing the way the world buys, sells and uses vehicles

“We use Databricks Workflows as our default orchestration tool to perform ETL and enable automation for about 300 jobs, of which approximately 120 are scheduled to run regularly.”

— Robert Hamlet, Lead Data Engineer, Enterprise Data Services, Cox Automotive

Cox Automotive Europe is part of Cox Automotive, the world’s largest automotive service organization, and is on a mission to transform the way the world buys, sells, owns and uses vehicles. They work in partnership with automotive manufacturers, fleets and retailers to improve performance and profitability throughout the vehicle lifecycle. Their businesses are organized around their customers’ core needs across vehicle solutions, remarketing, funding, retail and mobility. Their brands in Europe include Manheim, Dealer Auction, NextGear Capital, Modix and Codeweavers.

Cox’s enterprise data services team recently built a platform to consolidate the company’s data and enable their data scientists to create new data-driven products and services more quickly and easily. To enable their small engineering team to unify data and analytics on one platform while enabling orchestration and governance, the enterprise data services team turned to the Databricks Data Intelligence Platform, Workflows, Unity Catalog and Delta Sharing.

EASY ORCHESTRATION AND OBSERVABILITY IMPROVE ABILITY TO DELIVER VALUE

Cox Automotive's enterprise data services team maintains a data platform that primarily serves internal customers spanning across business units, though they also maintain a few data feeds to third parties. The enterprise data services team collects data from multiple internal sources and business units. "We use Databricks Workflows as our default orchestration tool to perform ETL and enable automation for about 300 jobs, of which approximately 120 are scheduled to run regularly," says Robert Hamlet, Lead Data Engineer, Enterprise Data Services, at Cox Automotive.

Jobs may be conducted weekly, daily or hourly. The amount of data processed in production pipelines today is approximately 720GB per day. Scheduled jobs pull from different areas both within and outside of the company. Hamlet uses Databricks Workflows to deliver data to the data science team, to the in-house data reporting team through Tableau, or directly into Power BI. "Databricks Workflows has a great user interface that allows you to quickly schedule any type of workflow, be it a notebook or JAR," says Hamlet. "Parametrization has been especially useful. It gives us clues as to how we can move jobs across environments. Workflows has all the features you would want from an orchestrator."

Hamlet also likes that Workflows provides observability into every workflow run and failure notifications so they can get ahead of issues quickly and troubleshoot before the data science team is impacted. "We use the job notifications feature to send failure notifications to a webhook, which is linked to our Microsoft Teams account," he says. "If we receive an alert, we go into Databricks to see what's going on. It's very useful to be able to peel into the run logs and see what errors occurred. And the Repair Run feature is nice to remove blemishes from your perfect history."

UNITY CATALOG AND DELTA SHARING IMPROVE DATA ACCESS ACROSS TEAMS

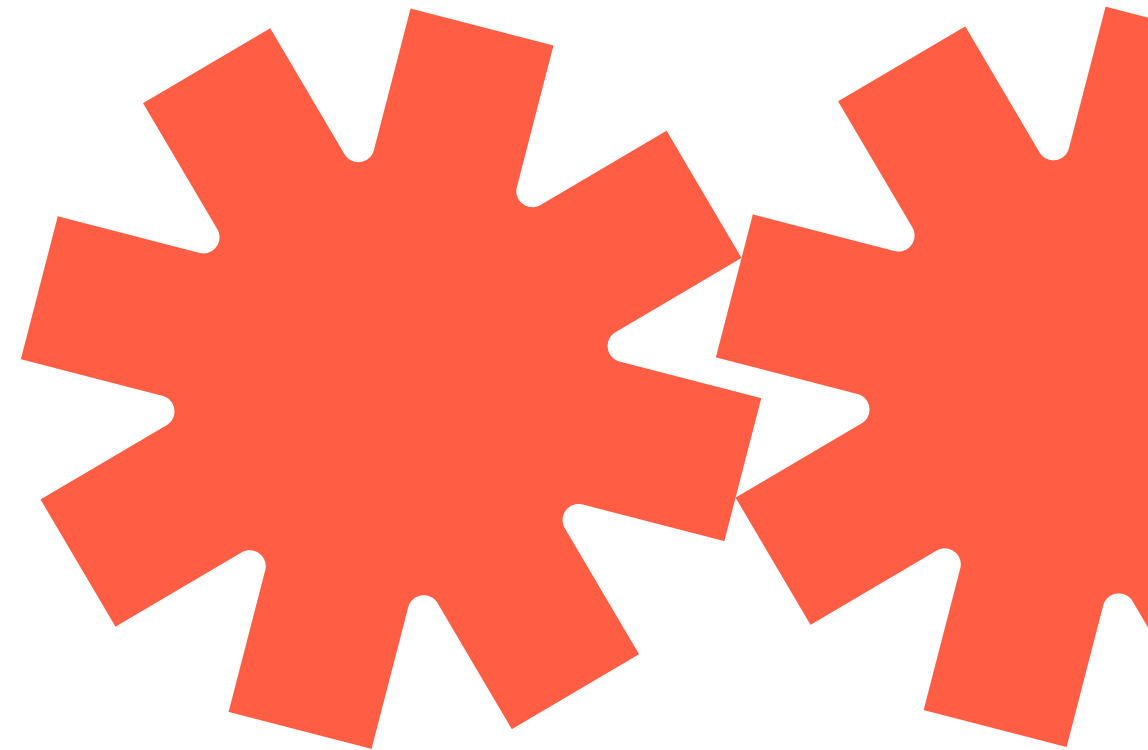
Hamlet's team recently began using Unity Catalog to manage data access, improving their existing method, which lacked granularity and was difficult to manage. "With our new workspace, we're trying to use more DevOps principles, infrastructure-as-code and groups wherever possible," he says. "I want to easily manage access to a wide range of data to multiple different groups and entities, and I want it to be as simple as possible for my team to do so. Unity Catalog is the answer to that."

The enterprise data services team also uses Delta Sharing, which natively integrates with Unity Catalog and allows Cox to centrally manage and audit shared data outside the enterprise data services team while ensuring robust security and governance. "Delta Sharing makes it easy to securely share data with business units and subsidiaries without copying or replicating it," says Hamlet. "It enables us to share data without the recipient having an identity in our workspace."

LOOKING AHEAD: INCORPORATING ADDITIONAL DATA INTELLIGENCE PLATFORM CAPABILITIES

Going forward, Hamlet plans to use Delta Live Tables (DLT) to make it easy to build and manage batch and streaming data pipelines that deliver data on the Databricks Data Intelligence Platform. DLT will help data engineering teams simplify ETL development and management. Eventually, Hamlet may also use Delta Sharing to easily share data securely with external suppliers and partners while meeting security and compliance needs. “DLT provides us an opportunity to make it simpler for our team. Scheduling Delta Live Tables will be another place we’ll use Workflows,” he says.

Hamlet is also looking forward to using the data lineage capabilities within Unity Catalog to provide his team with an end-to-end view of how data flows in the lakehouse for data compliance requirements and impact analysis of data changes. “That’s a feature I’m excited about,” Hamlet says. “Eventually, I hope we get to a point where we have all our data in the lakehouse, and we get to make better use of the tight integrations with things like data lineage and advanced permissions management.”





Block — building a world-class data platform

Block standardizes on Delta Live Tables to expand secure economic access for millions

90%

Improvement in
development velocity

150

Pipelines being onboarded in
addition to the 10 running daily

INDUSTRY

Financial Services

PLATFORM

Delta Live Tables, Data Streaming,
Machine Learning, ETL

CLOUD

AWS

Block is a global technology company that champions accessible financial services and prioritizes economic empowerment. Its subsidiaries, including Square, Cash App and TIDAL, are committed to expanding economic access. By utilizing artificial intelligence (AI) and machine learning (ML), Block proactively identifies and prevents fraud, ensuring secure customer transactions in real time. In addition, Block enhances user experiences by delivering personalized recommendations and using identity resolution to gain a comprehensive understanding of customer activities across its diverse services. Internally, Block optimizes operations through automation and predictive analytics, driving efficiency in financial service delivery. Block uses the Data Intelligence Platform to bolster its capabilities, consolidating and streamlining its data, AI and analytics workloads. This strategic move positions Block for the forthcoming automation-driven innovation shift and solidifies its position as a pioneer in AI-driven financial services.

ENABLING CHANGE DATA CAPTURE FOR STREAMING DATA EVENTS ON DELTA LAKE

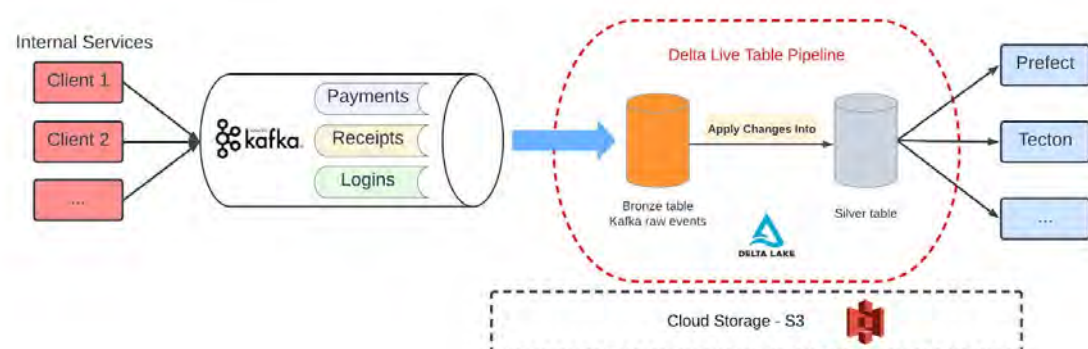
Block's Data Foundations team is dedicated to helping its internal customers aggregate, orchestrate and publish data at scale across the company's distributed system to support business use cases such as fraud detection, payment risk evaluation and real-time loan decisions. The team needs access to high-quality, low-latency data to enable fast, data-driven decisions and reactions.

Block had been consolidating on Kafka for data ingestion and Delta Lake for data storage. More recently, the company sought to make real-time streaming data available in Delta Lake as Silver (cleansed and conformed) data for analytics and machine learning. It also wanted to support event updates and simple data transformations and enable data quality checks to ensure higher-quality data. To accomplish this, Block considered a few alternatives, including the Confluent-managed Databricks Delta Lake Sink connector, a fully managed solution with low latency. However, that solution did not offer change data capture support and had limited transformation and data quality check support. The team also considered building their own solution with Spark Structured Streaming, which also provided low latency and strong data transformation capabilities. But that solution required the team to maintain significant code to define task workflows, change data and capture logic. They'd also have to implement their own data quality checks and maintenance jobs.

LEVERAGING THE LAKEHOUSE TO SYNC KAFKA STREAMS TO DELTA TABLES IN REAL TIME

Rather than redeveloping its data pipelines and applications on new, complex, proprietary and disjointed technology stacks, Block turned to the Data Intelligence Platform and **Delta Live Tables** (DLT) for change data capture and to enable the development of end-to-end, scalable streaming pipelines and applications. DLT pipelines simply orchestrate the way data flows between Delta tables for ETL jobs, requiring only a few lines of declarative code. It automates much of the operational complexity associated with running ETL pipelines and, as such, comes with preselected smart defaults yet is also tunable, enabling the team to optimize and debug easily. "DLT offers declarative syntax to define a pipeline, and we believed it could greatly improve our development velocity," says Yue Zhang, Staff Software Engineer for the Data Foundations team at Block. "It's also a managed solution, so it manages the maintenance tasks for us, it has data quality support, and it has advanced, efficient **autoscaling** and **Unity Catalog integration**."

Today, Block's Data Foundations team ingests events from internal services in Kafka topics. A DLT pipeline consumes those events into a Bronze (raw data) table in real time, and they use the DLT API to apply changes and merge data into a higher-quality Silver table. The Silver table can then be used by other DLT pipelines for model training, to schedule model orchestration, or to define features for a features store. "It's very straightforward to implement and build DLT pipelines," says Zhang.



The Block Data Foundations team's streaming data architecture with Delta Live Tables pipelines

Using the Python API to define a pipeline requires three steps: a row table, a Silver table and a merge process. The first step is to define the row table. Block consumes events from Kafka, performs some simple transformations, and establishes the data quality check and its rule. The goal is to ensure all events have a valid event ID.

The next step is to define the Silver table or target table, its storage location and how it is partitioned. With those tables defined, the team then determines the merge logic. Using the DLT API, they simply select **APPLY CHANGES INTO**. If two units have the same event ID, DLT will choose the one with the latest ingest timestamp. "That's all the code you need to write," says Zhang.

Finally, the team defines basic configuration settings from the DLT UI, such as characterizing clusters and whether the pipeline will run in continuous or triggered modes.

Following their initial DLT proof of concept, Zhang and his team implemented **CI/CD** to make DLT pipelines more accessible to internal Block teams. Different teams can now manage pipeline implementations and settings in their own repos, and, once they merge, simply use the Databricks pipelines API to create, update and delete those pipelines in the CI/CD process.

BOOSTING DEVELOPMENT VELOCITY WITH DLT

Implementing DLT has been a game-changer for Block, enabling it to boost development velocity. "With the adoption of Delta Live Tables, the time required to define and develop a streaming pipeline has gone from days to hours," says Zhang.

Meanwhile, managed maintenance tasks have resulted in better query performance, improved data quality has boosted customer trust, and more efficient autoscaling has improved cost efficiency. Access to fresh data means Block data analysts get more timely signals for analytics and decision-making, while Unity Catalog integration means they can better streamline and automate data governance processes. "Before we had support for Unity Catalog, we had to use a separate process and pipeline to stream data into S3 storage and a different process to create a data table out of it," says Zhang. "With Unity Catalog integration, we can streamline, create and manage tables from the DLT pipeline directly."

Block is currently running approximately 10 DLT pipelines daily, with about two terabytes of data flowing through them, and has another 150 pipelines to onboard. "Going forward, we're excited to see the bigger impacts DLT can offer us," adds Zhang.



Trek — global bicycle leader accelerates retail analytics

80%–90%

Acceleration in runtime of retail analytics solution globally

3X

Increase in daily data refreshes on Databricks

1 Week

Reduction in ERP data replication, which now happens in near real time

“How do you scale up analytics without blowing a hole in your technology budget? For us, the clear answer was to run all our workloads on Databricks Data Intelligence Platform and replicate our data in near real-time with Qlik.”

— Garrett Baltzer, Software Architect, Data Engineering, Trek Bicycle

Trek Bicycle started in a small Wisconsin barn in 1976, but their founders always saw something bigger. Decades later, the company is on a mission to make the world a better place to live and ride. Trek only builds products they love and provides incredible hospitality to customers as they aim to change the world for the better by getting more people on bikes. Frustrated by the rising costs and slow performance of their data warehouse, Trek migrated to Databricks Data Intelligence Platform. The company now uses Qlik to replicate their ERP data to Databricks in near real-time and stores data in Delta Lake tables. With Databricks and Qlik, Trek has dramatically accelerated their retail analytics to provide a better experience for their customers with a unified view of the global business to their data consumers, including business and IT users.

INDUSTRY

Retail and Consumer Goods

SOLUTION

Real-Time Point-of-Sale Analytics

PLATFORM

Delta Lake, Databricks SQL, Delta Live Tables, Data Streaming

PARTNER

Qlik

CLOUD

Azure



sales@3cloudsolutions.com // 888-88-AZURE // 3cloudsolutions.com

SLOW DATA PROCESSING HINDERS RETAIL ANALYTICS

As Trek Bicycle works to make the world better by encouraging more people to ride bikes, the company keeps a close eye on what's happening in their hundreds of retail stores. But until recently, running analytics on their retail data proved challenging because Trek relied on a data warehouse that couldn't scale cost-effectively.

"The more stores we added, the more information we added to our processes and solutions," explained Garrett Baltzer, Software Architect, Data Engineering, at Trek Bicycle. "Although our data warehouse did scale to support greater data volumes, our processing costs were skyrocketing, and processes were taking far too long. Some of our solutions were taking over 30 hours to produce analytics, which is unacceptable from a business perspective."

Adding to the challenge, Trek's data infrastructure hindered the company's efforts to achieve a global view of their business performance. Slow processing speeds meant Trek could only process data once per day for one region at a time.

"We were processing retail data separately for our North American, European and Asia-Pacific stores, which meant everyone downstream had to wait for actionable insights for different use cases," recalled Advait Raje, Team Lead, Data Engineering, at Trek Bicycle. "We soon made it a priority to migrate to a unified data platform that would produce analytics more quickly and at a lower cost."

DELTA LAKE UNIFIES RETAIL DATA FROM AROUND THE GLOBE

Seeking to modernize their data infrastructure to speed up data processing and unify all their data from global sources, Trek started migrating to the Databricks Data Intelligence Platform in 2019. The company's processing speeds immediately increased. Qlik's integration with the Databricks Data Intelligence Platform helps feed Trek's lakehouse. This replication allows Trek to build a wide range of valuable data products for their sales and customer service teams.

"Qlik enabled us to move relevant ERP data into Databricks where we don't have to worry about scaling vertically because it automatically scales parallel. Since 70 to 80% of our operational data comes from our ERP system, Qlik has made it possible to get far more out of our ERP data without increasing our costs," Baltzer explained.

Trek is now running all their retail analytics workloads in the Databricks Data Intelligence Platform. Today, Trek uses the Databricks Data Intelligence Platform to collect point-of-sale data from nearly 450 stores around the globe. All computation happens on top of Trek's lakehouse. The company runs a semantic layer on top of this lakehouse to power everything from strategic high-level reporting for C-level executives to daily sales and operations reports for individual store employees.

"Databricks Data Intelligence Platform has been a game-changer for Trek," said Raje. "With Qlik Cloud Data Integration on Databricks, it became possible to replicate relevant ERP data to our Databricks in real time, which made it far more accessible for downstream retail analytics. Suddenly, all our data from multiple repositories was available in one place, enabling us to reduce costs and deliver on business needs much more quickly."

Trek's BI and data analysts leverage **Databricks SQL**, their serverless data warehouse, for ad hoc analysis to answer business questions much more quickly. Internal customers can leverage Power BI connecting directly to Databricks to consume retail analytics data from Gold tables. This ease of analysis helps the company monitor and enhance their Net Promoter Scores. Trek uses Structured Streaming and Auto Loader functionality within Delta Live Tables to transform the data from Bronze to Silver or Gold, according to the medallion architecture.

"Delta Live Tables have greatly accelerated our development velocity," Raje reported. "In the past, we had to use complicated ETL processes to take data from raw to parsed. Today, we just have one simple notebook that does it, and then we use Delta Live Tables to transform the data to Silver or Gold as needed."

DATA INTELLIGENCE PLATFORM ACCELERATES ANALYTICS BY 80% TO 90%

By moving their data processing to the Databricks Data Intelligence Platform and integrating data with Qlik, Trek has dramatically increased the speed of their processing and overall availability of data. Prior to implementing Qlik, they had a custom program that, once a week, on a Sunday, replicated Trek's ERP data from on-premises servers to a data lake using bulk copies. Using Qlik, Trek now replicates relevant data from their ERP system as Delta tables directly in their lakehouse.

"We used to work with stale ERP data all week because replication only happened on Sundays," Raje remarked. "Now we have a nearly up-to-the-minute view of what's going on in our business. That's because Qlik lets us keep replicating through the day, streaming data from ERP into our lakehouse."

Trek's retail analytics solution used to take 48+ hours to produce meaningful results. Today, Trek runs the solution on the Databricks Data Intelligence Platform to get results in six to eight hours — an 80 to 90% improvement, thus allowing daily runs. A complementary retail analytics solution went from 12–14 hours down to under 4–5 hours, thereby enabling the lakehouse to refresh three times per day, compared to only once a day previously.

"Before Databricks, we had to run our retail analytics once a day on North American time, which meant our other regions got their data late," said Raje. "Now, we refresh the lakehouse three times per day, one for each region, and stakeholders receive fresh data in time to drive their decisions. Based on the results we've achieved in the lakehouse, we're taking a Databricks-first approach to all our new projects. We're even migrating many of our on-premises BI solutions to Databricks because we're all-in on the lakehouse."

"Databricks Data Intelligence Platform, along with data replication to Databricks using Qlik, aligns perfectly with our broader cloud-first strategy," said Steve Novoselac, Vice President, IT and Digital, at Trek Bicycle. "This demonstrates confidence in the adoption of this platform at Trek."



INDUSTRY

Financial Services

SOLUTION

Financial Crimes Compliance,
Customer Profile Scoring, Financial
Reconciliation, Credit Risk
Reporting, Synthesizing Multiple
Data Sources, Data Sharing and
Collaboration

PLATFORM

Delta Lake, ETL, Delta Sharing, Data
Streaming, Databricks SQL

CLOUD

Azure



Coastal Community Bank — mastering the modern data platform for exponential growth

< 10

Minutes to securely share large
datasets across organizations

99%

Decrease in processing
duration (2+ days to 30 min.)

12X

Faster partner onboarding by
eliminating sharing complexity

“We’ve done two years’ worth of work here in nine months. Databricks enables access to a single source of truth and our ability to process high volumes of transactions that gives us confidence we can drive our growth as a community bank and a leading banking-as-a-service provider.”

— Curt Queyrouze, President, Coastal Community Bank

Many banks continue to rely on decades-old, mainframe-based platforms to support their back-end operations. But banks that are modernizing their IT infrastructures and integrating the cloud to share data securely and seamlessly are finding they can form an increasingly interconnected financial services landscape. This has created opportunities for community banks, fintechs and brands to collaborate and offer customers more comprehensive and personalized services. Coastal Community Bank is headquartered in Everett, Washington, far from some of the world’s largest financial centers. The bank’s CCBX division offers banking as a service (BaaS) to financial technology companies and broker-dealers. To provide personalized financial products, better risk oversight, reporting and compliance, Coastal turned to the Databricks Data Intelligence Platform and Delta Sharing, an open protocol for secure data sharing, to enable them to share data with their partners while ensuring compliance in a highly regulated industry.

sales@3cloudsolutions.com // 888-88-AZURE // 3cloudsolutions.com

LEVERAGING TECH AND INNOVATION TO FUTURE-PROOF A COMMUNITY BANK

Coastal Community Bank was founded in 1997 as a traditional brick-and-mortar bank. Over the years, they grew to 14 full-service branches in Washington state offering lending and deposit products to approximately 40,000 customers. In 2018, the bank's leadership broadened their vision and long-term growth objectives, including how to scale and serve customers outside their traditional physical footprint. Coastal leaders took an innovative step and launched a plan to offer BaaS through CCBX, enabling a broad network of virtual partners and allowing the bank to scale much faster and further than they could via their physical branches alone.

Coastal hired Barb MacLean, Senior Vice President and Head of Technology Operations and Implementation, to build the technical foundation required to help support the continued growth of the bank. "Most small community banks have little technology capability of their own and often outsource tech capabilities to a core banking vendor," says MacLean. "We knew that story had to be completely different for us to continue to be an attractive banking-as-a-service partner to outside organizations."

To accomplish their objectives, Coastal would be required to receive and send vast amounts of data in near real-time with their partners, third parties and the variety of systems used across that ecosystem. This proved to be a challenge as most banks and providers still relied on legacy technologies and antiquated processes like once-a-day batch processing. To scale their BaaS offering, Coastal needed a better way to manage and share data. They also required a solution that could scale while ensuring that the highest levels of security, privacy and strict compliance requirements were met. "The list of things we have to do to prove that we can safely and soundly operate as a regulated financial institution is ever-increasing," says MacLean. "As we added more customers and therefore more customer information, we needed to scale safely through automation."

Coastal also needed to accomplish all this with their existing small team. "As a community bank, we can't compete on a people basis, so we have to have technology tools in place that teams can learn easily and deploy quickly," adds MacLean.

TACKLING A COMPLEX DATA ENVIRONMENT WITH DELTA SHARING

With the goal of having a more collaborative approach to community banking and banking as a service, Coastal began their BaaS journey in January 2023 when they chose Cavallo Technologies to help them develop a modern, future-proof data platform to support their stringent customer data sharing and compliance requirements. This included tackling infrastructure challenges, such as data ingestion complexity, speed, data quality and scalability. "We wanted to use our small, nimble team to our advantage and find the right technology to help us move fast and do this right," says MacLean.

“We initially tested several vendors, however learned through those tests we needed a system that could scale for our needs going forward,” says MacLean. Though very few members of the team had used Databricks before, Coastal decided to move from a previously known implementation pattern and a data lake-based platform to a lakehouse approach with Databricks. The lakehouse architecture addressed the pain points they experienced using a data lake-based platform, such as trying to sync batch and streaming data. The dynamic nature and changing environments of Coastal’s partners required handling changes to data structure and content. The Databricks Data Intelligence Platform provided resiliency and tooling to deal with both data and schema drift cost-effectively at scale.

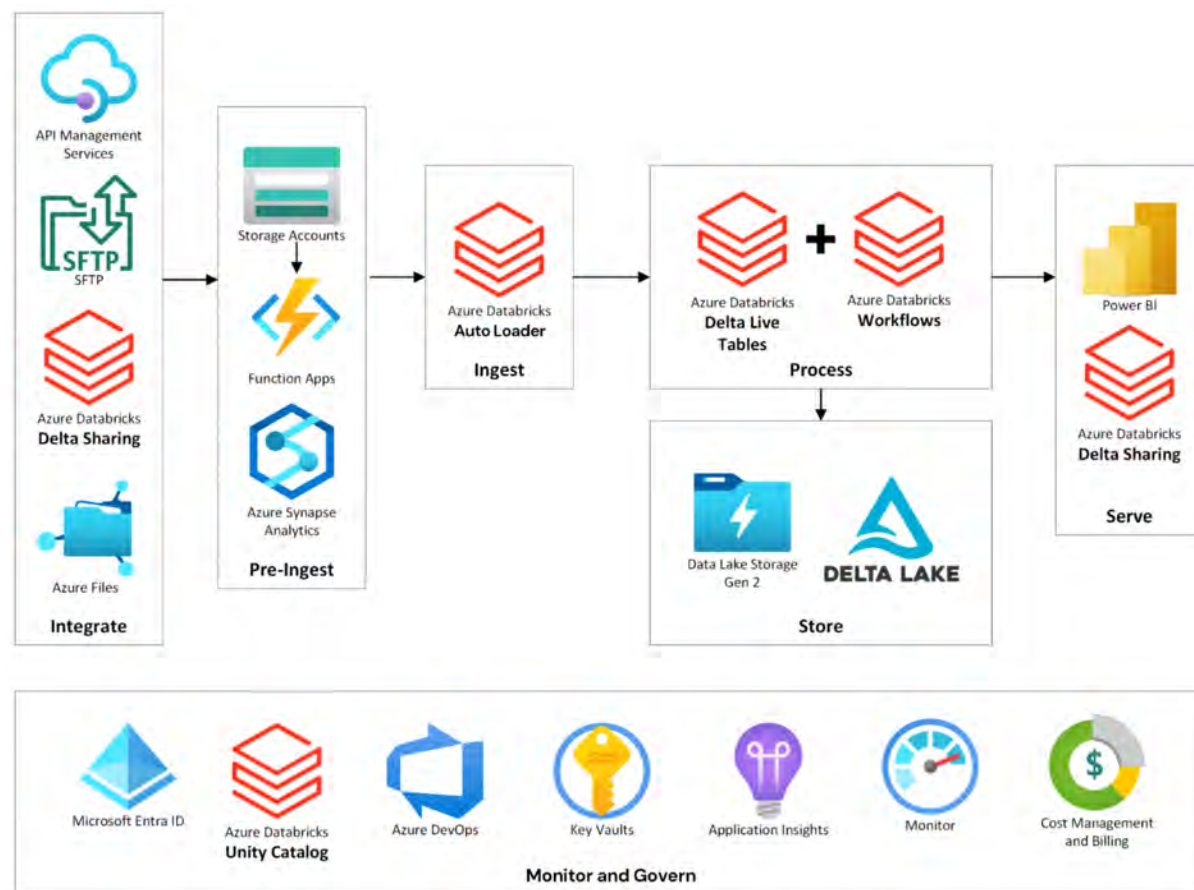
Coastal continued to evolve and extend their use of Databricks tools, including Auto Loader, Structured Streaming, Delta Live Tables, Unity Catalog and Databricks repos for CI/CD, as they created a robust software engineering practice for data at the bank. Applying software engineering principles to data can often be neglected or ignored by engineering teams, but Coastal knew that it was critical to managing the scale and complexity of the internal and external environment in which they were working. This included having segregated environments for development, testing and production, having technical leaders approve the promotion of code between environments, and include data privacy and security governance.

Coastal also liked that Databricks worked well with Azure out of the box. And because it offered a consolidated toolkit for data transformation and engineering, Databricks helped address any risk concerns. “When you have a highly complex technical environment with a myriad of tools, not only inside your own environment but in our partners’ environments that we don’t control, a consolidated toolkit reduces complexity and thereby reduces risk,” says MacLean.

Initially, MacLean’s team evaluated several cloud-native solutions, with the goal of moving away from a 24-hour batch world and into real-time data processing since any incident could have wider reverberations in a highly interconnected financial system. “We have all these places where data is moving in real time. What happens when someone else’s system has an outage or goes down in the middle of the day? How do you understand customer and bank exposure as soon as it happens? How do we connect the batch world with the real-time world? We were trapped in a no-man’s-land of legacy, batch-driven systems, and partners are too,” explains MacLean.

“We wanted to be a part of a community of users, knowing that was the future, and wanted a vendor that was continually innovating,” says MacLean. Similarly, MacLean’s team evaluated the different platforms for ETL, BI, analytics and data science, including some already in use by the bank. “Engineers want to work with modern tools because it makes their lives easier ... working within the century in which you live. We didn’t want to Frankenstein things because of a wide toolset,” says MacLean. “Reducing complexity in our environment is a key consideration, so using a single platform has a massive positive impact. Databricks is the hands-down winner in apples-to-apples comparisons to other tools like Snowflake and SAS in terms of performance, scalability, flexibility and cost.”

MacLean explained that Databricks included everything, such as Auto Loader, repositories, monitoring and telemetry, and cost management. This enabled the bank to benefit from robust software engineering practices so they could scale to serving millions of customers, whether directly or via their partner network. MacLean explained, “We punch above our weight, and our team is extremely small relative to what we’re doing, so we wanted to pick the tools that are applicable to any and all scenarios.”



The Databricks Data Intelligence Platform has greatly simplified how Coastal and their vast ecosystem of financial service partners securely share data across data platforms, clouds or regions.

IMPROVING TIME TO VALUE AND GROWING THEIR PARTNER NETWORK

In the short time since Coastal launched CCBX, it has become the bank's primary customer acquisition and growth division, enabling them to grow BaaS program fee income by 32.3% year over year. Their use of Databricks has also helped them achieve unprecedented time to value. "We've done two years' worth of work here in nine months," says Curt Queyrouze, President at Coastal.

Almost immediately, Coastal saw exponential improvements in core business functions. "Activities within our risk and compliance team that we need to conduct every few months would take 48 hours to execute with legacy inputs," says MacLean. "Now we can run those in 30 minutes using near real-time data."

Despite managing myriad technology systems, Databricks helps Coastal remove barriers between teams, enabling them to share live data with each other safely and securely in a matter of minutes so the bank can continue to grow quickly through partner acquisition. "The financial services industry is still heavily reliant on legacy, batch-driven systems, and other data is moving in real time and needs to be understood in real time. How do we marry those up?" asks MacLean. "That was one of the fundamental reasons for choosing Databricks. We have not worked with any other tool or technology that allows us to do that well."

CCBX leverages the power and scale of a network of partners. Delta Sharing uses an open source approach to data sharing and enables users to share live data across platforms, clouds and regions with strong security and governance. Using Delta Sharing meant Coastal could manage data effectively even when working with partners and third parties using inflexible legacy technology systems. “The data we were ingesting is difficult to deal with,” says MacLean. “How do we harness incoming data from about 20 partners with technology environments that we don’t control? The data’s never going to be clean. We decided to make dealing with that complexity our strength and take on that burden. That’s where we saw the true power of Databricks’ capabilities. We couldn’t have done this without the tools their platform gives us.”

Databricks also enabled Coastal to scale from 40,000 customers (consumers and small-medium businesses in the north Puget Sound region) to approximately 6 million customers served through their partner ecosystem and dramatically increase the speed at which they integrate data from those partners. In one notable case, Coastal was working with a new partner and faced the potential of having to load data on 80,000 customers manually. “We pointed Databricks at it and had 80,000 customers and the various data sources ingested, cleaned and prepared for our business teams to use in two days,” says MacLean. “Previously, that would have taken one to two months at least. We could not have done that with any prior existing tool we tried.”

With Delta Sharing on Databricks, Coastal now has a vastly simplified, faster and more secure platform for onboarding new partners and their data. “When we want to launch and grow a product with a partner, such as a point-of-sale consumer loan, the owner of the data would need to send massive datasets on tens of thousands of customers. Before, in the traditional data warehouse approach, this would typically take one to two months to ingest new data sources, as the schema of the sent data would need to be changed in order for our systems to read it. But now we point Databricks at it and it’s just two days to value,” shares MacLean.

While Coastal’s data engineers and business users love this improvement in internal productivity, the larger transformation has been how Databricks has enabled Coastal’s strategy to focus on building a rich partner network. They now have about 20 partners leveraging different aspects of Coastal’s BaaS.

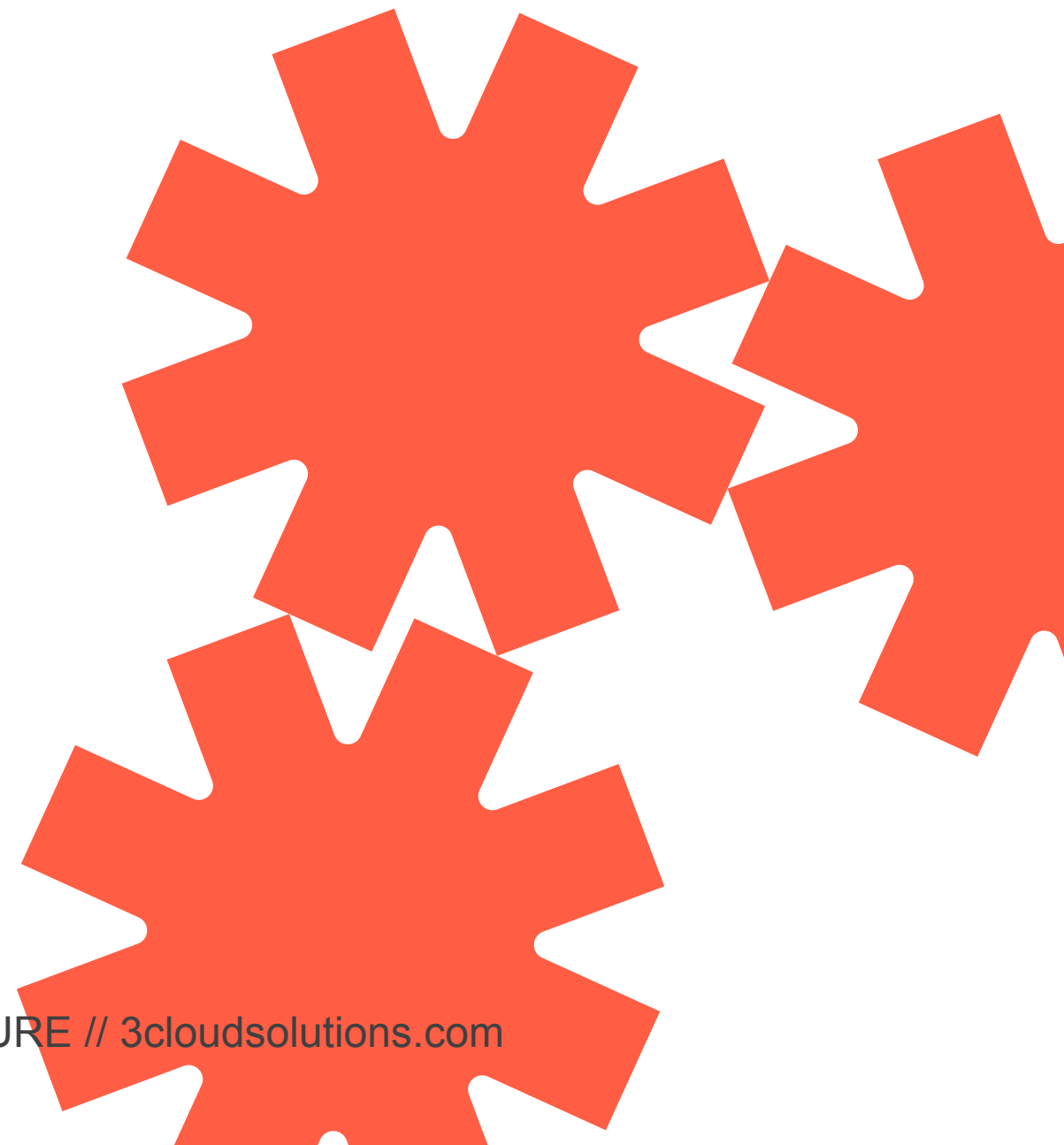
Recently, Coastal’s CEO had an ask about a specific dataset. Based on experience from their previous data tools, they brought in a team of 10 data engineers to comb through the data, expecting this to be a multiday or even multi-week effort. But when they actually got into their Databricks Data Intelligence Platform, using data lineage on Unity Catalog, they were able to give a definitive answer that same afternoon. MacLean explains that this is not an anomaly. “Time and time again, we find that even for the most seemingly challenging questions, we can grab a data engineer with no context on the data, point them to a data pipeline and quickly get the answers we need.”

The bank's use of Delta Sharing has also allowed Coastal to achieve success with One, an emerging fintech startup. One wanted to sunset its use of Google BigQuery, which Coastal was using to ingest One's data. The two organizations needed to work together to find a solution. Fortunately, One was also using Databricks. "We used Delta Sharing, and after we gave them a workspace ID, we had tables of data showing up in our Databricks workspace in under 10 minutes," says MacLean. (To read more about how Coastal is working with One, [read the blog](#).) MacLean says Coastal is a leader in skills, technology and modern tools for fintech partners.

DATA AND AI FOR GOOD

With a strong data foundation set, MacLean has a larger vision for her team. "Technologies like generative AI open up self-serve capabilities to so many business groups. For example, as we explore how to reduce financial crimes, if you are taking a day to do an investigation, that doesn't scale to thousands of transactions that might need to be investigated," says MacLean. "How do we move beyond the minimum regulatory requirements on paper around something like anti-money laundering and truly reduce the impact of bad actors in the financial system?"

For MacLean this is about aligning her organization with Coastal's larger mission to use finance to do better for all people. Said MacLean, "Where are we doing good in terms of the application of technology and financial services? It's not just about optimizing the speed of transactions. We care about doing better on behalf of our fellow humans with the work that we do."





Powys Teaching Health Board — improving decision-making to save lives faster

< 1 year To modernize data infrastructure	40% Decrease in time to insight	65% More productive with Databricks Assistant
--	---	---

“The adoption of Databricks has ensured that we can future-proof our data capabilities. It has transformed and modernized the way we work, and that has a direct impact on the quality of care delivered to our community.”

— Jake Hammer, Chief Data Officer, Powys Teaching Health Board (PTHB)

The inability to access complete and high-quality data can have a direct impact on a healthcare system’s ability to deliver optimal patient outcomes. Powys Teaching Health Board (PTHB), serving the largest county in Wales, is responsible for planning and providing national health services for approximately a quarter of the country. However, roughly 50% of the data they need to help inform patient-centric decisions doesn’t occur within Powys and is provided by neighboring organizations in varying formats, slowing their ability to connect data with the quality of patient care. Converting all this data — from patient activity (e.g., appointments) and workforce data (e.g., schedules) — to actionable insights is difficult when it comes in from so many disparate sources. PTHB needed to break down these silos and make it easier for nontechnical teams to access the data. With the Databricks Data Intelligence Platform, PTHB now has a unified view of all their various data streams, empowering healthcare systems to make better decisions that enhance patient care.

- INDUSTRY
Healthcare and Life Sciences
- SOLUTION
Forward-Looking Intelligence
- PLATFORM
Data Intelligence Platform,
Unity Catalog
- CLOUD
Azure

SILOS AND SYSTEM STRAIN HINDER DATA-DRIVEN INSIGHTS

The demand for PTHB's services has increased significantly over the years as they've dealt with evolving healthcare needs and population growth. As new patients enter the national healthcare system, so does the rise in data captured about the patient, hospital operations and more. With this rapid influx of data coming from various hospitals and healthcare systems around the country, PTHB's legacy system began to reach its performance and scalability limits, quickly developing data access and ingestion bottlenecks that not only wasted time, but directly impacted patient care. And as the diversity of data rose, their legacy system buckled under the load.

"Our data sat in so many places that it caused major frustrations. Our on-premises SQL warehouse couldn't cope with the scale of our growing data estate," explained Jake Hammer, Chief Data Officer at PTHB. "We needed to move away from manually copying data between places. Finding a platform that would allow us to take advantage of the cloud and was flexible enough to safeguard our data within a single view for all to easily access was critical."

How could PTHB employees make data-driven decisions if the data was hard to find and difficult to understand? Hammer realized that they needed to first modernize their data infrastructure in the cloud and migrate to a platform capable of unifying their data and making it readily available for downstream analytics use cases: from optimizing staff schedules to providing actionable insights for clinicians so they can provide timely and targeted care. Hammer's team estimated that it would take five to 10 years to modernize their tech stack in this way if they were to follow their own processes and tech stack. But they needed a solution now. Enter Databricks.

IMPROVING DATA DEMOCRATIZATION WITH A UNIFIED PLATFORM

PTHB chose the Databricks Data Intelligence Platform to house all new incoming data, from any source. This includes the data for a large number of low-code apps (e.g., Power Apps) so that Hammer's team can now work with data that was historically kept on paper — making it significantly easier for people to access and analyze the data at scale.

Data governance is also critical, but creating standard processes was difficult before transitioning to Databricks as their core platform. With Unity Catalog, PTHB has a model where all of their security and governance is done only once at the Databricks layer. "The level of auditing in Databricks gives us a high level of assurance. We need to provide different levels of access to many different individuals and systems," added Hammer. "Having a tool that enables us to confidently manage this complex security gives both ourselves and our stakeholders assurance. We can more easily and securely share data with partners."

Deriving actionable insights on data through numerous Power BI dashboards with ease is something PTHB could not do before. "Now HR has the data they need to improve operational efficiency while protecting the bottom line," said Hammer. "They can self-serve any necessary data, and they can see where there are gaps in rosters or inefficiencies in on-the-ground processes. Being able to access the right data at the right time means they can be smarter with rostering, resource management and scheduling."

FEDERATED LAKEHOUSE IMPROVES TEAM EFFICIENCY AND REAL-TIME DATA ENHANCES PATIENT CARE

With the Databricks Data Intelligence Platform, PTHB has taken their first step toward modernization by moving to the cloud in less than a year — a much quicker timeline than their 10-year estimate — and providing a federated lakehouse to unify all their data. Through the lakehouse, they are able to seamlessly connect to their on-premises SQL warehouse and remote BigQuery environment at NHS Wales to create a single view of their data estate. “With the Databricks Platform, and by leveraging features such as Lakehouse Federation to integrate remote data, PTHB data practitioners now work from a single source of truth to improve decision-making and patient outcomes,” explained Hammer.

From an operational standpoint, the impact of a modern platform has been significant, with efficiencies skyrocketing to an estimated 40% time savings in building data pipelines for analytics. They also estimate spending 65% less time answering questions from business data users with the help of Databricks Assistant. This AI-powered tool accelerated training for PTHB, helping traditional SQL staff embrace new programming languages and empowering them to be more productive without overreliance on the data engineering team.

A modern stack and newfound efficiencies throughout the data workflow has fueled Hammer’s ability to expand into more advanced use cases. “Data science was an area we simply hadn’t thought about,” explained Hammer. “Now we have the means to explore predictive use cases like forecasting feature utilization.”

Most importantly, PTHB has a solution that’s future-proof and can tackle any data challenge, which is critical given how they are seeing a rapidly growing environment of APIs, adoption of open standards and new sources of real-time data. “I can trust our platform is future-proof, and I’m probably the only health board to be able to say that in Wales at the moment,” said Hammer. “Just like with the data science world, the prediction world was something we never thought was possible. But now we have the technology to do anything we put our minds and data to.”

Tens of millions of production workloads run daily on Databricks

Easily ingest and transform batch and streaming data on the [Databricks Data Intelligence Platform](#). Orchestrate reliable production workflows while Databricks automatically manages your infrastructure at scale. Increase the productivity of your teams with built-in data quality testing and support for software development best practices.

About Databricks

Databricks is the data and AI company. More than 10,000 organizations worldwide — including Block, Comcast, Condé Nast, Rivian, Shell and over 60% of the Fortune 500 — rely on the Databricks Data Intelligence Platform to take control of their data and put it to work with AI. Databricks is headquartered in San Francisco, with offices around the globe, and was founded by the original creators of Lakehouse, Apache Spark™, Delta Lake and MLflow. To learn more, follow Databricks on [LinkedIn](#), [X](#) and [Facebook](#).



Accelerate Your Azure Journey with 3Cloud

3Cloud is the premier pure-play Azure partner in the ecosystem with unparalleled expertise in all things Azure. We specialize in delivering top-tier Azure infrastructure, cutting-edge AI, robust data & analytics and ground-breaking app development. Leveraging our extensive experience, advanced tools and customized accelerators, we ensure the quickest time to value for your Azure-based projects.

*Microsoft Partner Top Honors since 2017. Visit 3cloudsolutions.com for a full list of awards.



sales@3cloudsolutions.com // 888-88-AZURE // 3cloudsolutions.com